How to build AI product sense

The secret is using Cursor for non-technical work (inside: 75 free days of Cursor Pro this out!)

TAL RAVIV AND AMAN KHAN
FEB 03, 2026 • PAID

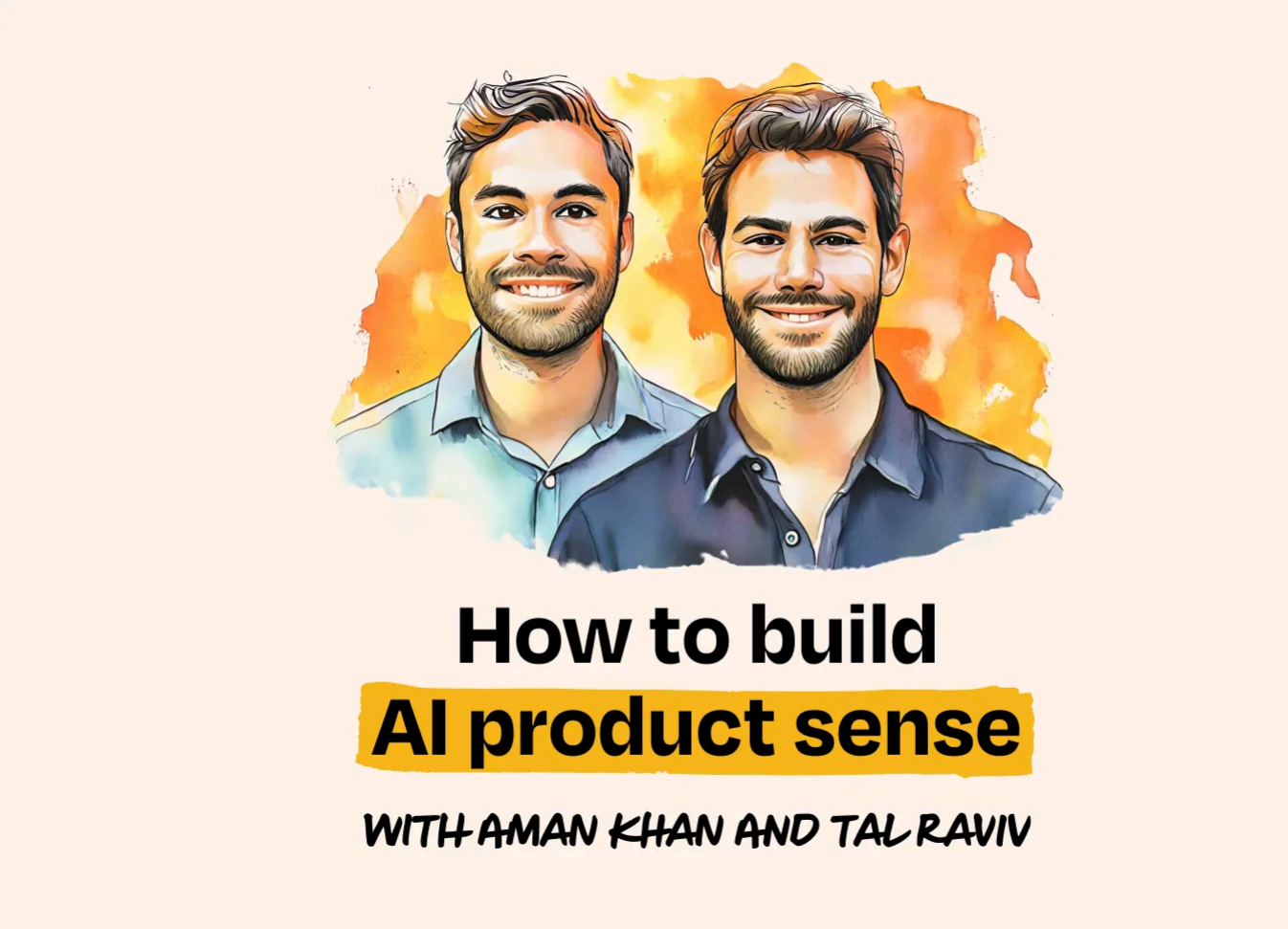
👋 *Hey there, I'm Lenny. Each week, I answer reader questions about building product, growth, and accelerating your career. For more: Lenny's Podcast | How I AI | Lennybot | Favorite AI and PM courses | Favorite public speaking course*

P.S. ***Insider subscribers*** ***get a free year*** *of Lovable, Manus, Replit, Gamma, n8n, Canva, ElevenLabs, Amp, Factory, Devin, Bolt, Wispr Flow, Linear, PostHog, Framer, Railway, Granola, Warp, Perplexity, Magic Patterns, Mobbin, ChatPRD, and Stripe Atlas. Yes, th for real.* ***Learn more****.*

The post you're about to read took over 100 hours to create. That's because it's n post. It's an open-source interactive AI experience that will help you build AI pr sense. Tal and Aman ran dozens of usability sessions, wrote evals, optimized eac prompt you'll find below, and even partnered with Cursor to get you free credits below!) so that you can try this at home. I've never seen anything like what they' together, and I'm excited to bring it to you.

Next week, in part two of this series, you'll learn how to take this newfound AI intuition and apply it to your own product.

For more from Tal and Aman, check out their in-depth workshop Build AI Product Sens



You're in a product meeting and someone mentions "subagents" or "context engineering" or "agent memory." You nod along. You know what these terms me you're just hoping no one expects you to use them in a sentence.

You've watched the video explainers, bookmarked the infographics, vibe coded a apps, and even shipped an AI feature. So why does it still feel like you're miles fr truly understanding all this stuff?

We (Tal and Aman) have both been there, over and over again while building AI products for tens of thousands of customers. The problem isn't you. The problem the "AI hype industrial complex." Most AI content is designed to induce FOMO,

We found that the single most transformative habit to internalize important A concepts was to move away from consumer-grade UIs (ChatGPT, Granola, Lova and into more powerful AI coding agents like Cursor and Claude Code. Getting hands dirty with coding agents has helped us build our "AI product sense"—the to correctly anticipate what will be truly impactful for users and also feasible wit

AI product sense is encountering support tickets about AI "forgetting" facts and recognizing it as context rot. Or watching a user struggle through a workflow an confidently saying that agent memory solves this—and knowing how to restructu the experience.

We've learned more about how AI products actually work in the past three mont using Cursor for daily, non-technical tasks than in three years of using ChatGPT is because coding agents transparently show their work. You can read AI's reaso inspect the tool calls, and watch the context window fill up. You hit the same wa engineers building AI applications, naturally intuit your own solutions, and start anticipating trends and industry announcements.

We now spend our days using Cursor and Claude Code for daily work: strategy, prioritization, decision-making, data analysis, and productivity. They serve as ou thinking partner and personal operating system.

In this post, we'll guide you through using AI coding agents for your non-techni product work:

- In steps 1-4 we'll **get set up and familiar with Cursor** with a fun Disney-ther exercise
- In steps 5-6 we'll **use Cursor to get hands-on with choosing AI models and calling tools**
- In steps 7-10, **we'll build a lightweight personal OS** (i.e. your own AI produc

sense.

Step 1: Download Cursor

Cursor is hands-down the best coding agent to most quickly ramp up your AI pr sense.

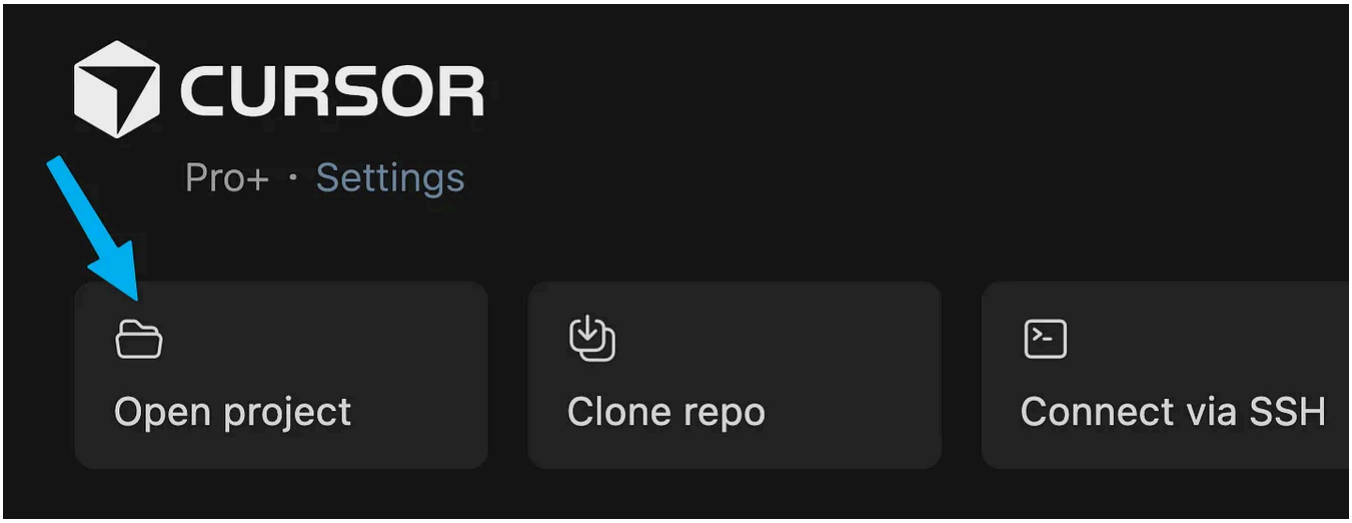
You're probably hearing about Claude Code all over, and we love it for delegating running independent tasks like vibe coding. Cursor is still our favorite for *pairing* AI and being able to directly watch an AI agent at work.

Cursor is a visual, clickable user experience and can be used with a variety of AI providers, including OpenAI and Anthropic. That means you can very likely use work.

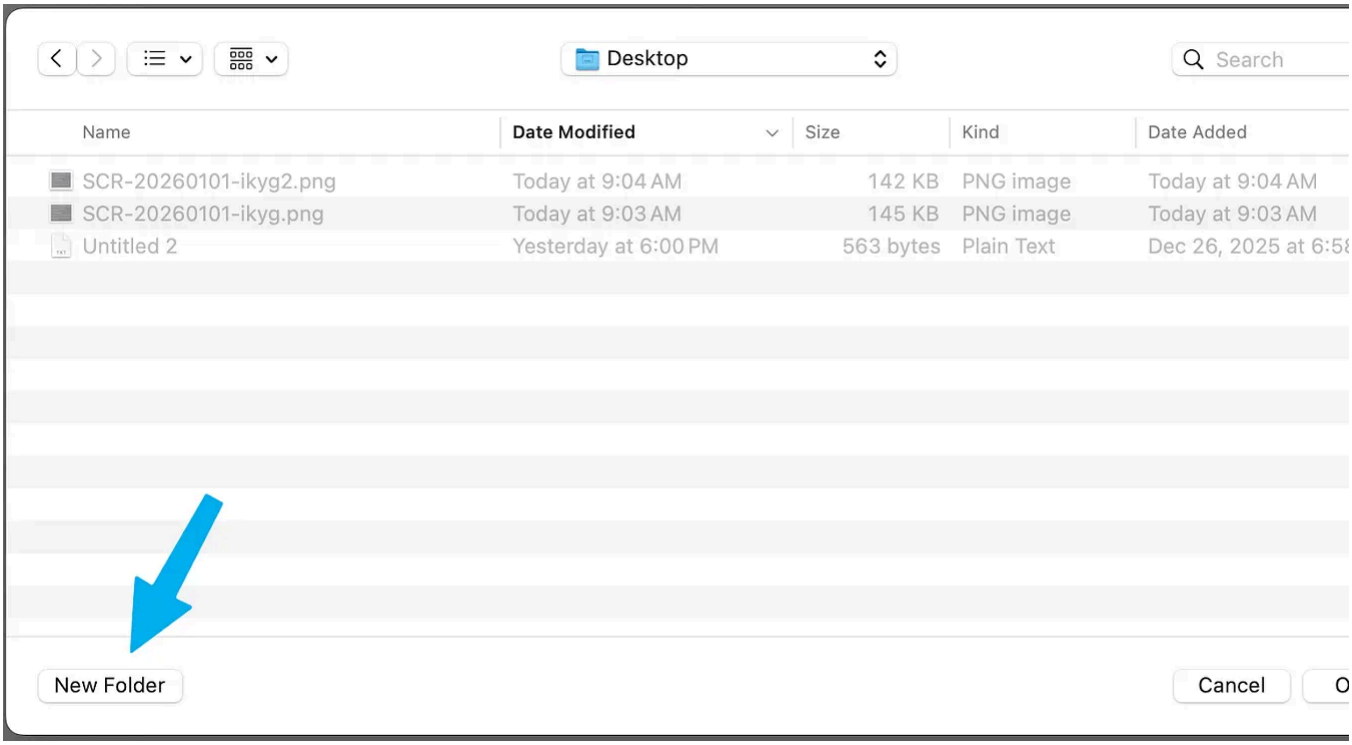
Downloading Cursor will take you two minutes. **Do it right now!**

1. Download and install Cursor. **For this post, make sure to download and instal desktop app, not the web version of Cursor.**

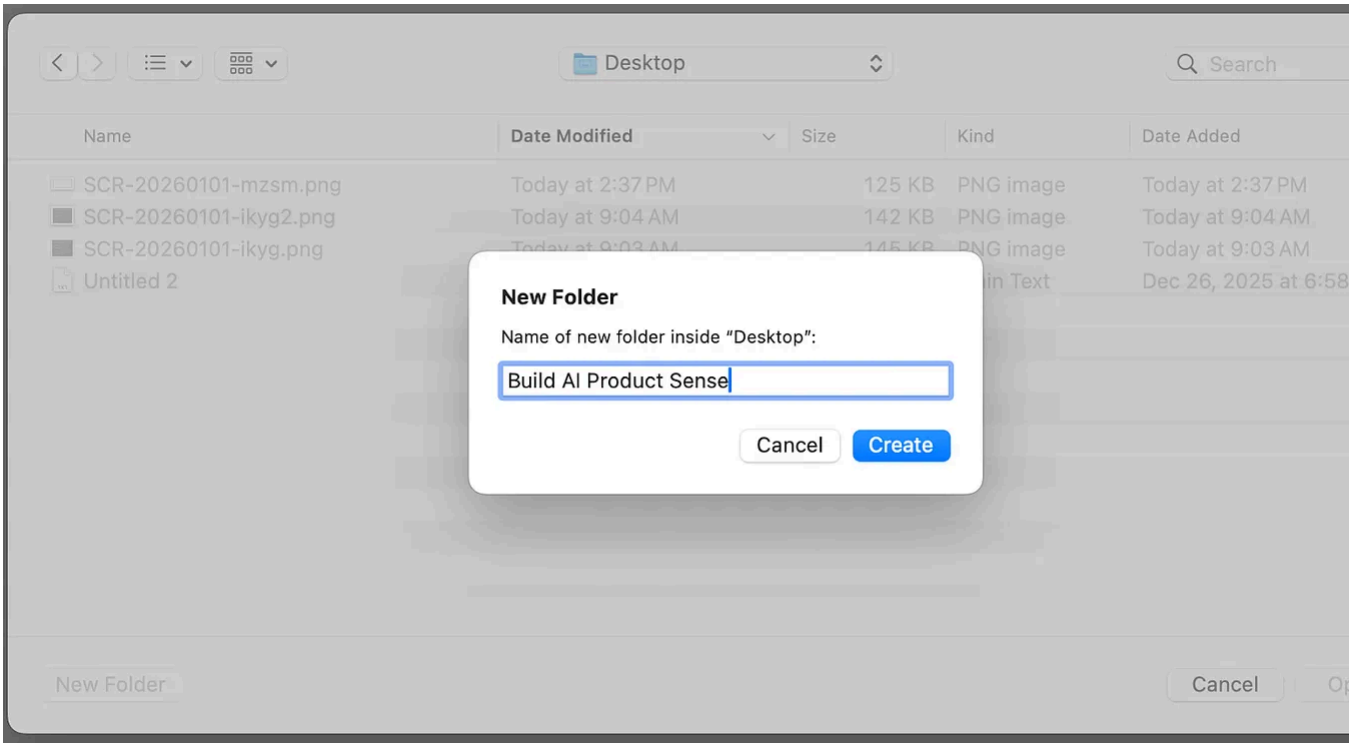
Step 2: Create a new project



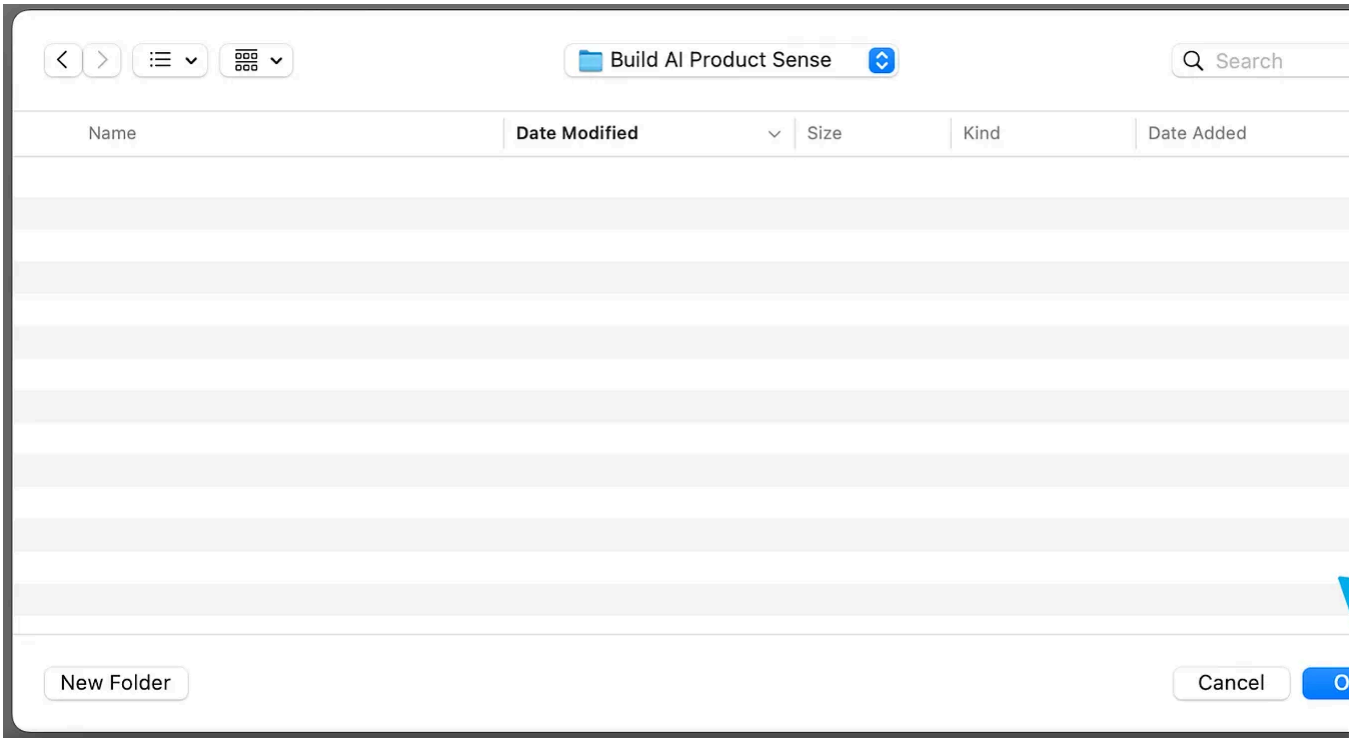
Click “New Folder”:



Name it “Build AI Product Sense” and click “Create”:



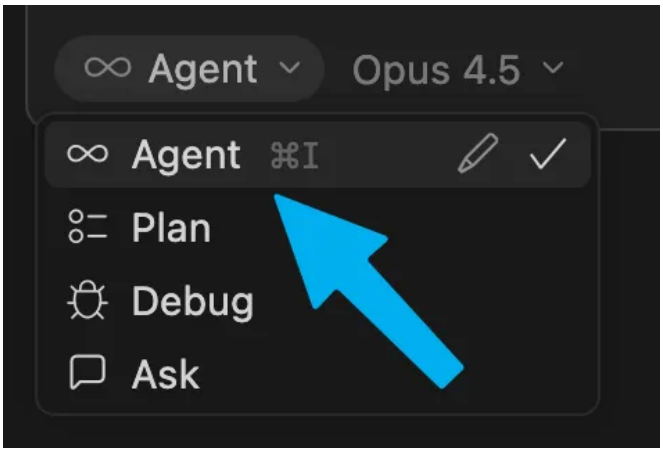
And finally, click “Open” (yes, it’s unusual to click Open on an empty folder, but do it, it’ll work):



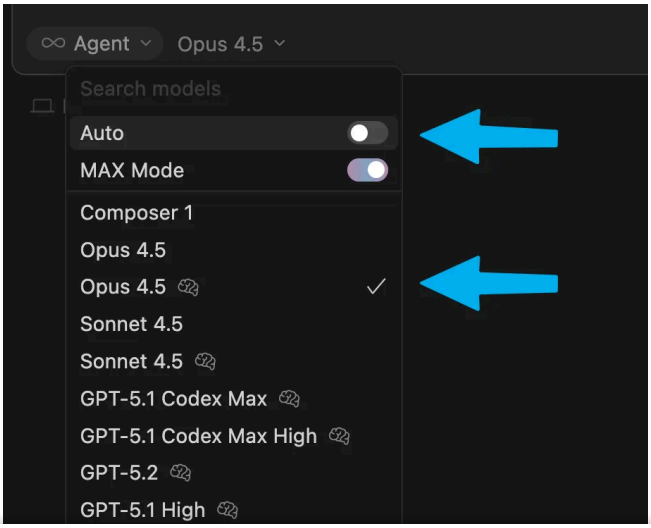
Strap in, because you're going to continue the experience inside Cursor itself, ins by the children's science show *The Magic School Bus*.

If you don't have time to ride the Magic School Bus right now, you can keep reading belo However, to build your AI product sense, we recommend you come back and try continu this post inside Cursor using the prompt below.

Make sure you're in "Agent" mode. This allows Cursor to take actions (such as fe this post from the internet).



In the "model" dropdown, turn off "auto" and select *Opus 4.5* 🧠:



🎁 Side note: We're hooking you up w free Cursor credits 🎁

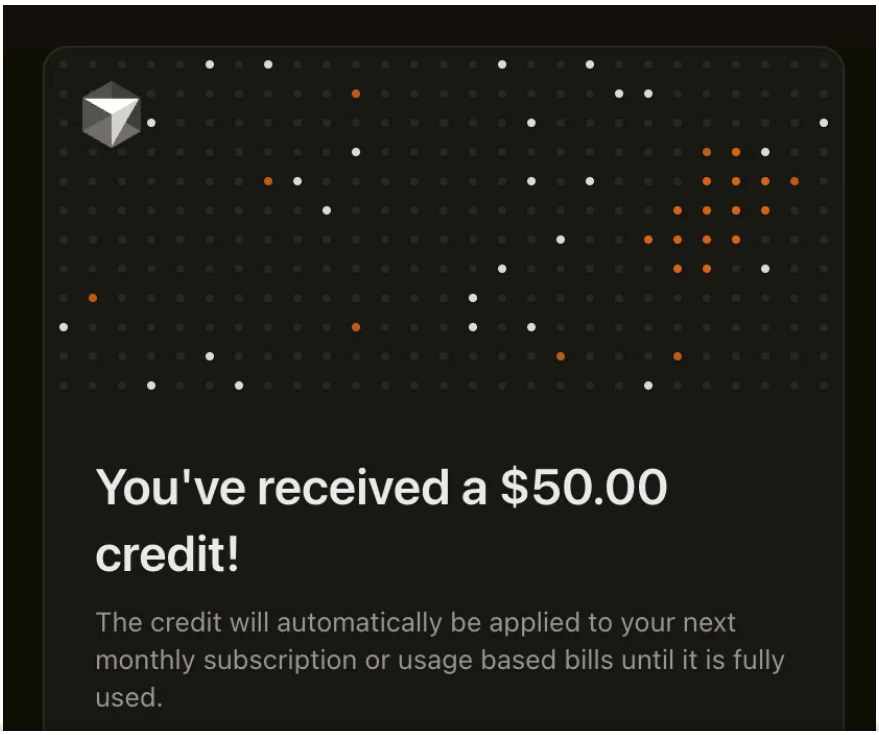
To help you experience the full power of this tutorial, we're hooking up Lenny's Newsletter subscribers with $50 in free Cursor credit. This is enough to get you 1 months of standard usage. A huge thank-you to Ben Lang and team Cursor for m this happen. Note: Supplies are limited, so we may run out of free codes. Act fast

How to grab your free Cursor credits:

1. Visit Cursor.com/dashboard and sign up for an account.

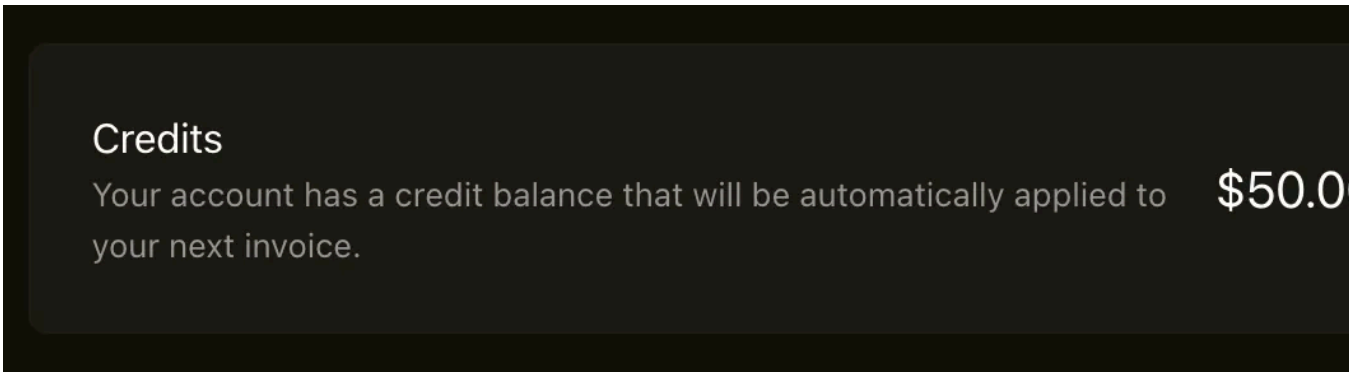
2. Become an annual (or Insider) Lenny's Newsletter subscriber.

3. Claim your free Cursor code (scroll to the bottom to find Cursor), click the but redeem your code, and you'll see the screen below:

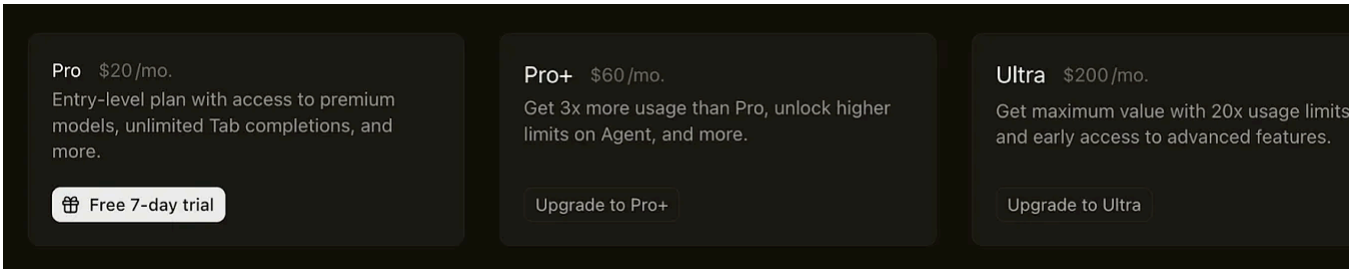


4. Click “Get Started” to apply the credits to your account. [*Credits can be redeeme both free and paid accounts.*]

5. Once you’ve redeemed the credits, you should see this box appear in your Curs dashboard. [*If you don’t see the credits, try to hard-refresh, or log out and log back in. I doesn’t work, shoot a message to hi@cursor.com and mention this post.*]

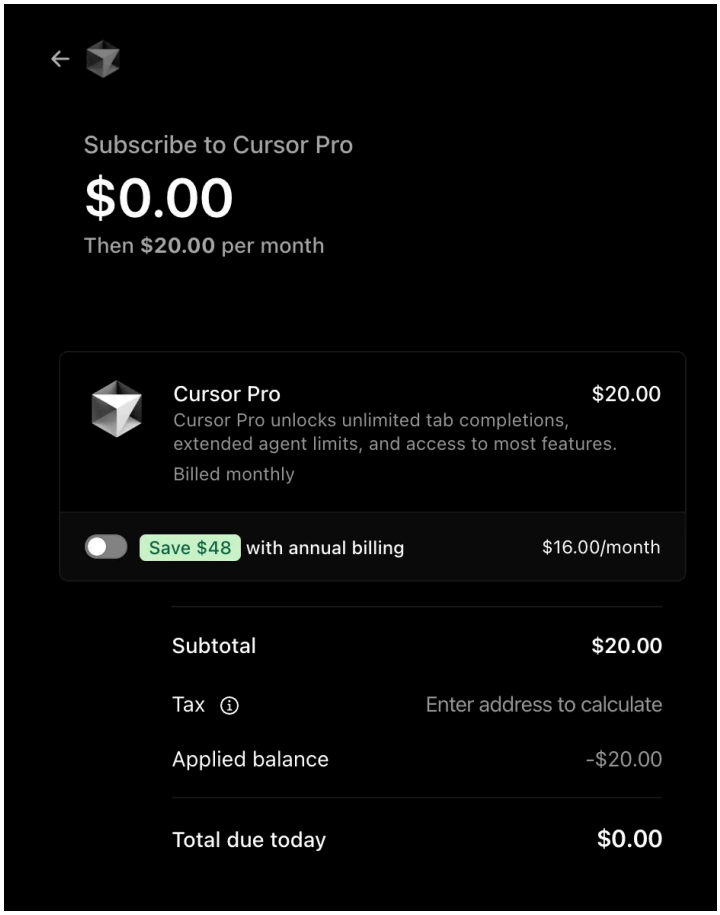


6. Finally, you’ll need to upgrade to the Pro (or higher) plan to use the latest AI m like Opus 4.5. If you’re already on the Pro or higher plan, you’re all set.



Once you see the credits in your Dashboard, go ahead and upgrade. You won’t be charged anything (you have 2.5 months’ worth of credits). The credits will be automatically applied to the next invoice (and can also be applied to “on-demand usage” if you enable it).

It should look like this:



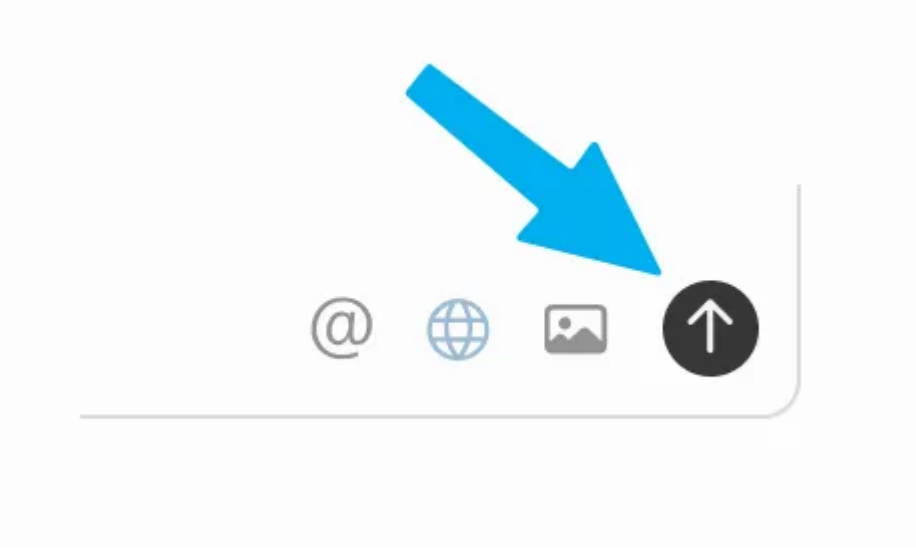
Now, paste the prompt below into the Cursor chat box (c just click this link):

> Help me build my AI product sense using this post (that I have not yet read): https://buildaiproductsense.com/magicschoolbus. (Do not o it in a browser — that will be distracting — use cURL or other tool.)
>
> Start by giving me an overview of why we're here and wher we're going with this, so I feel super-motivated to stick with it. Then pause and confirm I'm ready to start. Use t pause to learn more about my professional context (less a

> You are both a really good 1-1 tutor for hands-on learnin
> AND the Cursor agent. Have me take action so I'm engaged
> learning. Ask me one question at a time. Before starting
> steps/stages/ideas/concepts, stop and check in with me an
> encourage me to explain it back to you — and hold me to a
> high bar — like an effective, empathetic tutor.
>
> It's important that you cover every single concept contai
> in this post, in sequential order. Keep me motivated by
> signposting and giving clarity on how much we've done and
> much is left. (That said, leave room to follow my curiosi
> and go off script, as long as overall we are progressing
> through the post.)
>
> Use the original words of the post when relevant (you hav
> permission to use them as your words in first person, rat
> than explicitly quoting someone else). Sentence-case your
> headings (not title case).
>
> Anytime you come across an image inline in the post, read
> image (one at a time, just in time, not in advance, stori
> temporarily if needed). This is important to understand t
> contents of the post.
>
> We are already talking inside a Cursor chat thread, so le
> use this same thread for as much as we can. Important: Yo
> are also the Cursor agent! So when I say a prompt that yo
> suggested, or give a task like "change this file," act on

> Remember that Cursor might be configured in a lot of different ways visually and is constantly evolving, so av assumptions about where a UI element might be. The file explorer may be on the left or the right.
>
> Anytime you try to use a tool of any kind, it's going to for my approval, and that's going to feel scary. So I nee you to explain why you're asking and why it's safe to approve. It might even be a teachable moment – you can ti the goal of the post (and where we are in the journey) th well, you're an agent and this is you in action!
>
> Consistently encourage me to use the voice recording feat (a 🎙️ icon under the chat box) to build the habit of spee to-text.

And click "submit":



AI will walk you through the rest of this post. Seatbelts, everyone! We'll see you i
Cursor.

Step 3: Cursor may look intimidating, but you're more familiar with it than you realize

Cursor looks *Matrix*-style geeky, but it's just ChatGPT, a text editor, and a file ex
smooshed into one window.

We repeat, Cursor is just three tools you've used plenty of times before, combir

1. ChatGPT
2. A text editor
3. File explorer

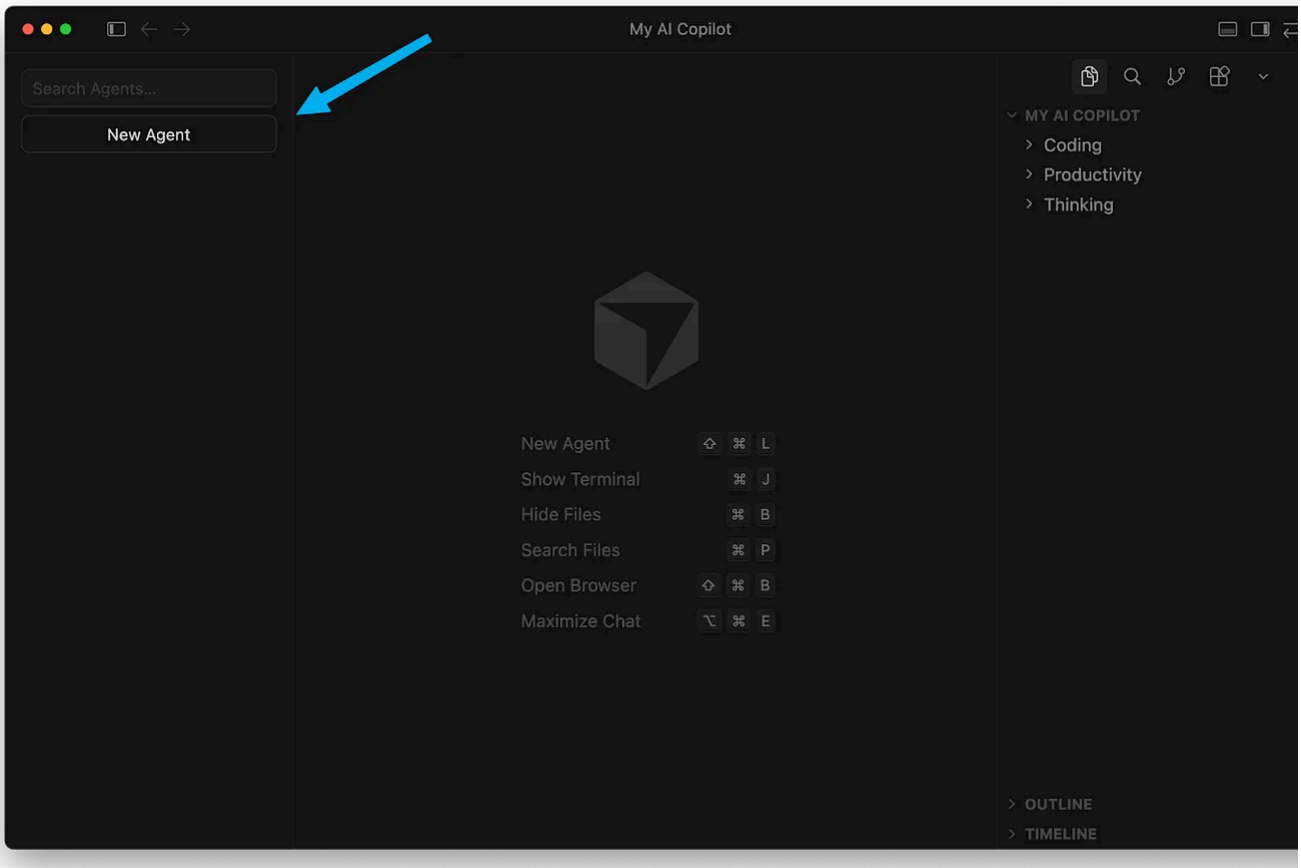
One of Tal's students, a salesperson, said it best: Cursor is "AI that can touch any
on my computer."

Here's a quick tour:

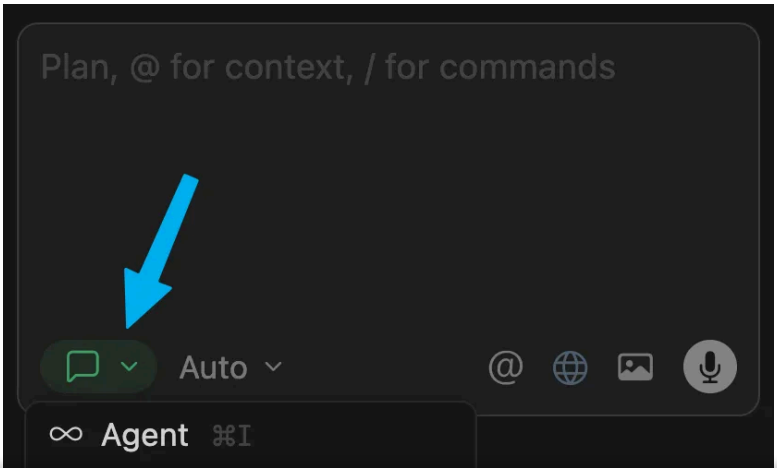
1. Agents

Agents are a fancy term for "chats." This is where you'll interact with AI.

On the left, you'll see an empty panel that will contain your agent history. Click '
agent" to start your first chat ("agent" is synonymous with "chat thread"):



You'll see a familiar chat box. Click on the dropdown in the bottom left corner. Y can select between "Ask" mode and "Agent" mode (ignore the other options, like and Debug, for now).

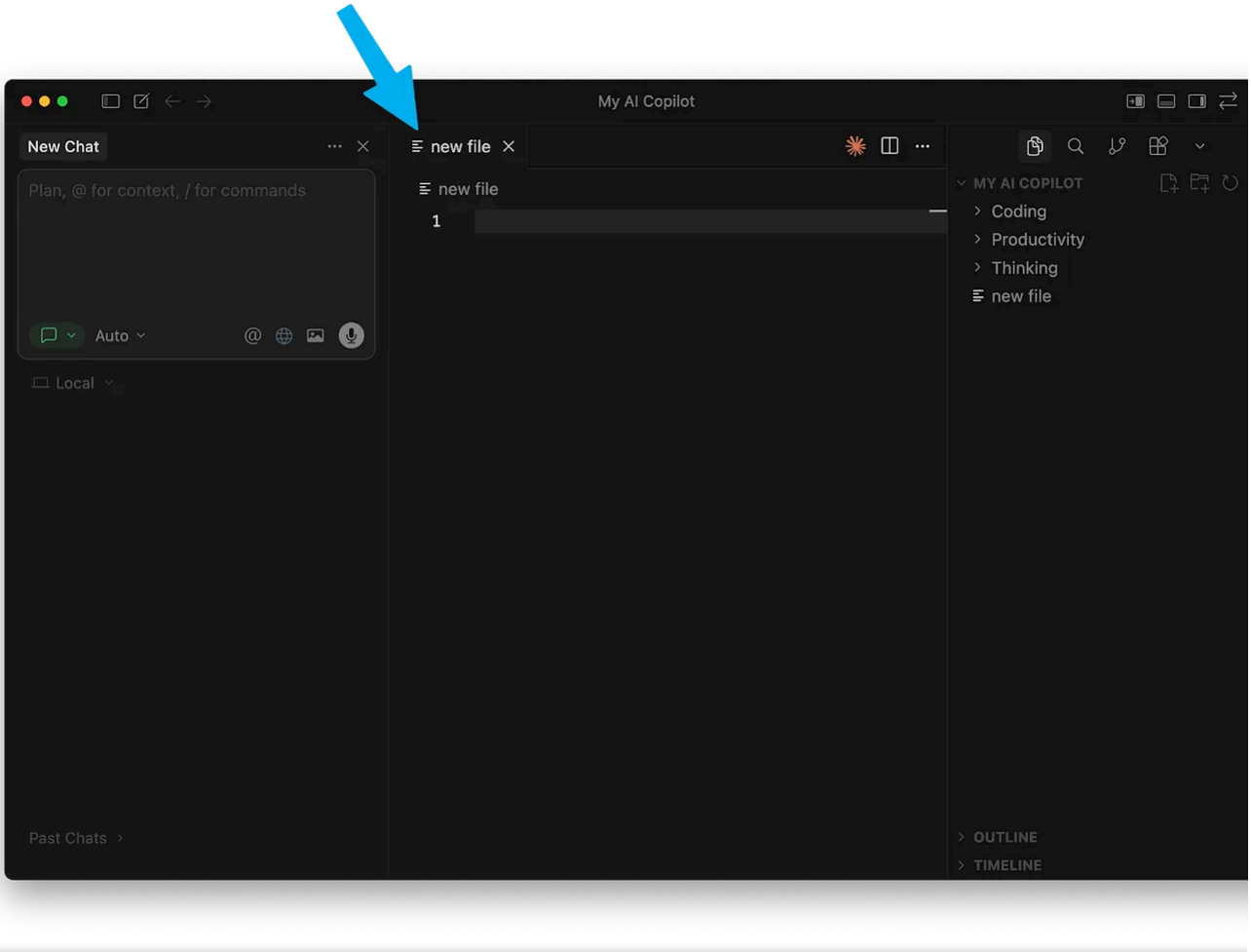


“Ask” mode is using Cursor just like classic ChatGPT: for chatting, and not maki any changes. This is great for brainstorming or asking questions, before taking a action. You can immediately start using this instead of standard ChatGPT/Claud

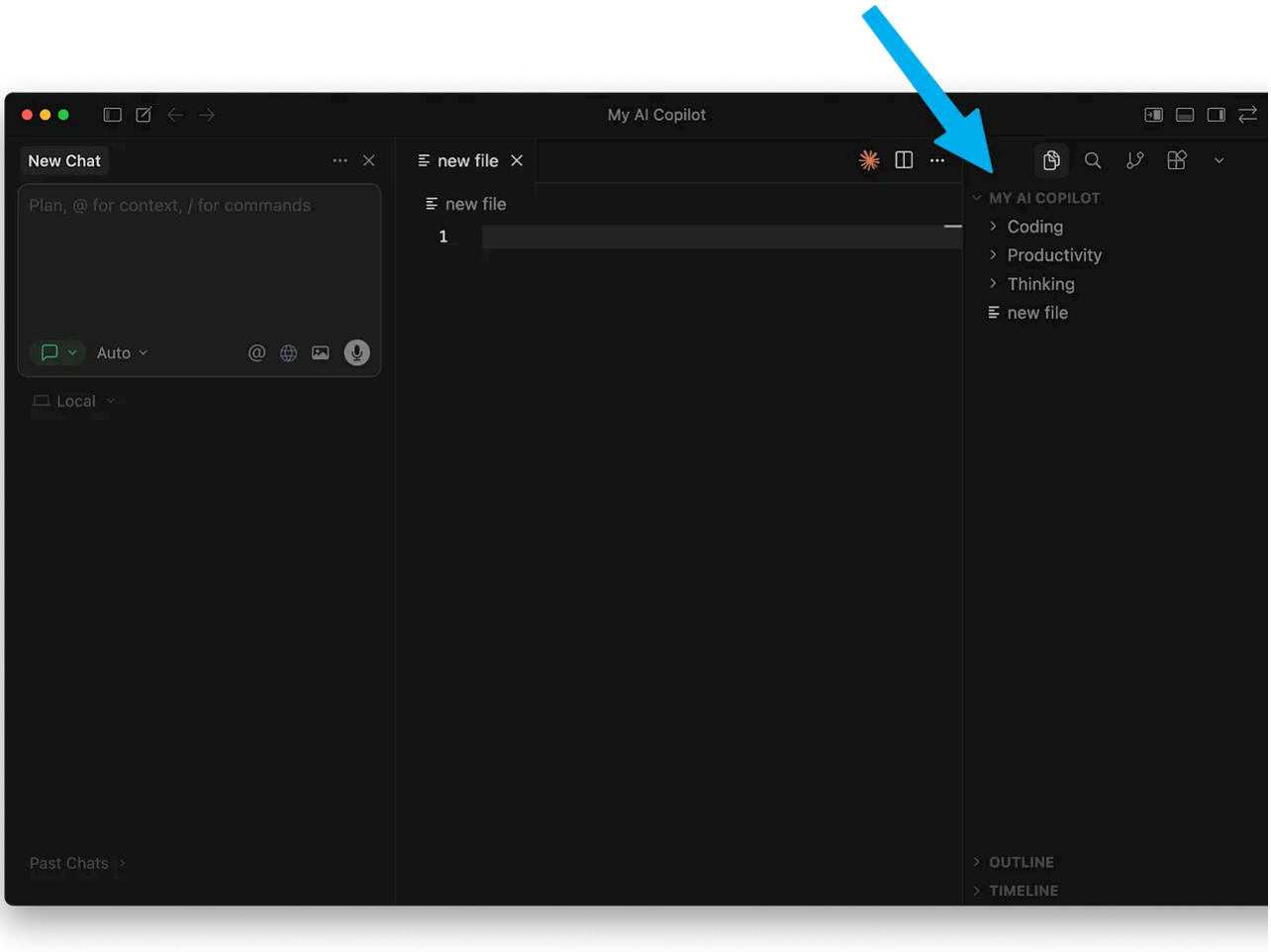
“Agent” mode is for when we want Cursor to modify files in our project. We’ll us together in a moment.

2. Editor

We’ll use this panel to view and manually edit text files. This is the same as using Edit on a Mac or Notepad on Windows. To see the text editor, you might have to a new file or double-click on an existing file.



The file explorer shows all the files and folders in your project. This is the same a Finder on a Mac or File Explorer on Windows, and it may be on the right or left c Cursor depending on the latest version. To expand it, you might have to click the icon at the top of the window to make it visible. (You can always use Ctrl + B on Windows or Cmd + B on a Mac.)



Even if Cursor feels intimidating at first, the core concepts are the same as with LLM you've already used. Cursor just has a bit more configurability, options, and things you can play with. This is your playground to build intuition.

Cursor is our choice for getting real work done, not just learning AI

important for us to say that regardless of understanding technical concepts, we r
spend most of our days in coding agents for non-technical tasks.

So what's the practical difference, and why did we make the switch? First, Curso
fundamentally the same idea as ChatGPT projects: files as knowledge, chat as
interface, and instructions that always apply.

Two small form-factor differences change everything:

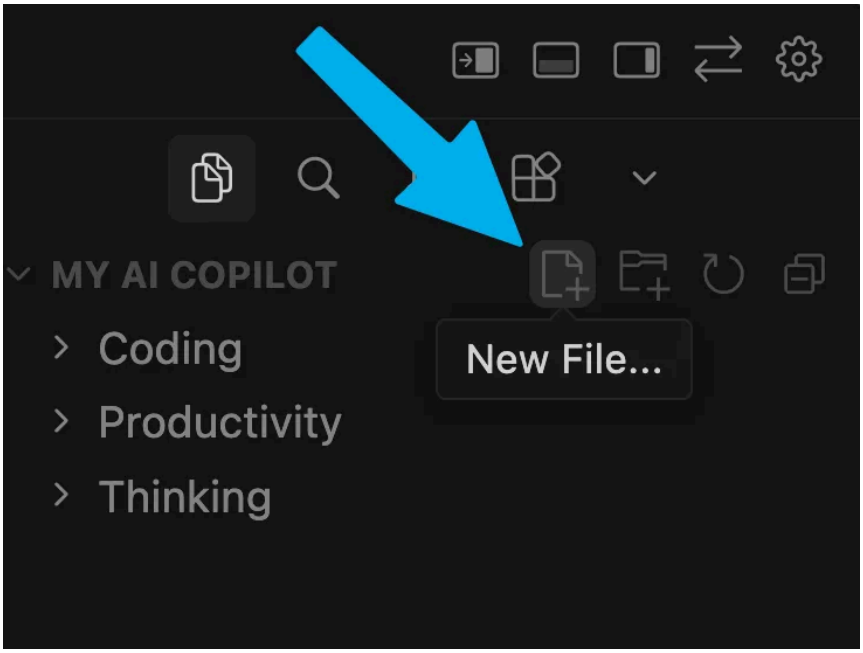
- You drag and drop specific files/folders into each chat (selective context)
- The AI edits your files directly (malleable knowledge)

These create a tight loop where every chat automatically improves your project
knowledge (but only when you tell the agent to do so). In ChatGPT projects, hist
and outputs live in long chats. You manually copy things back to project knowle
Cursor, outputs live in documents and chats become disposable one-offs because
value lives in documents, not in conversation history.

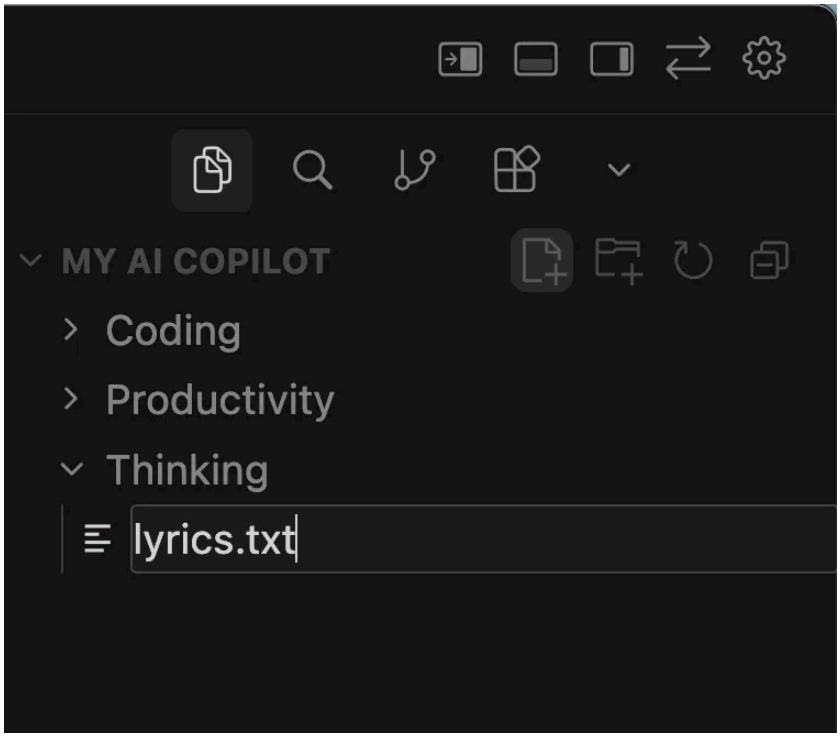
The main takeaway here is that your knowledge base will cover more ground, an
update more frequently, because you're using the personal OS to build and edit c
every single day.

Step 4: Create a Disney song parody to learn the basi of Cursor

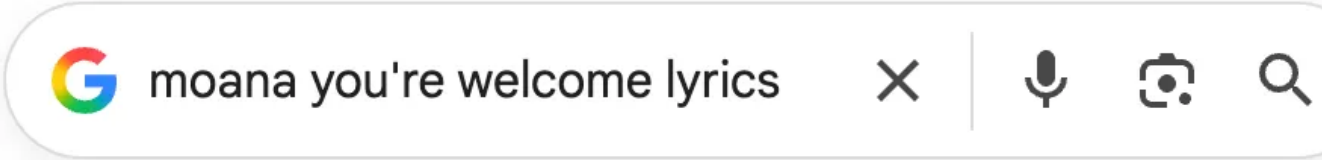
We'll start by creating a new blank file in Cursor. Hover your mouse in the file
explorer, and click the "New File" button:



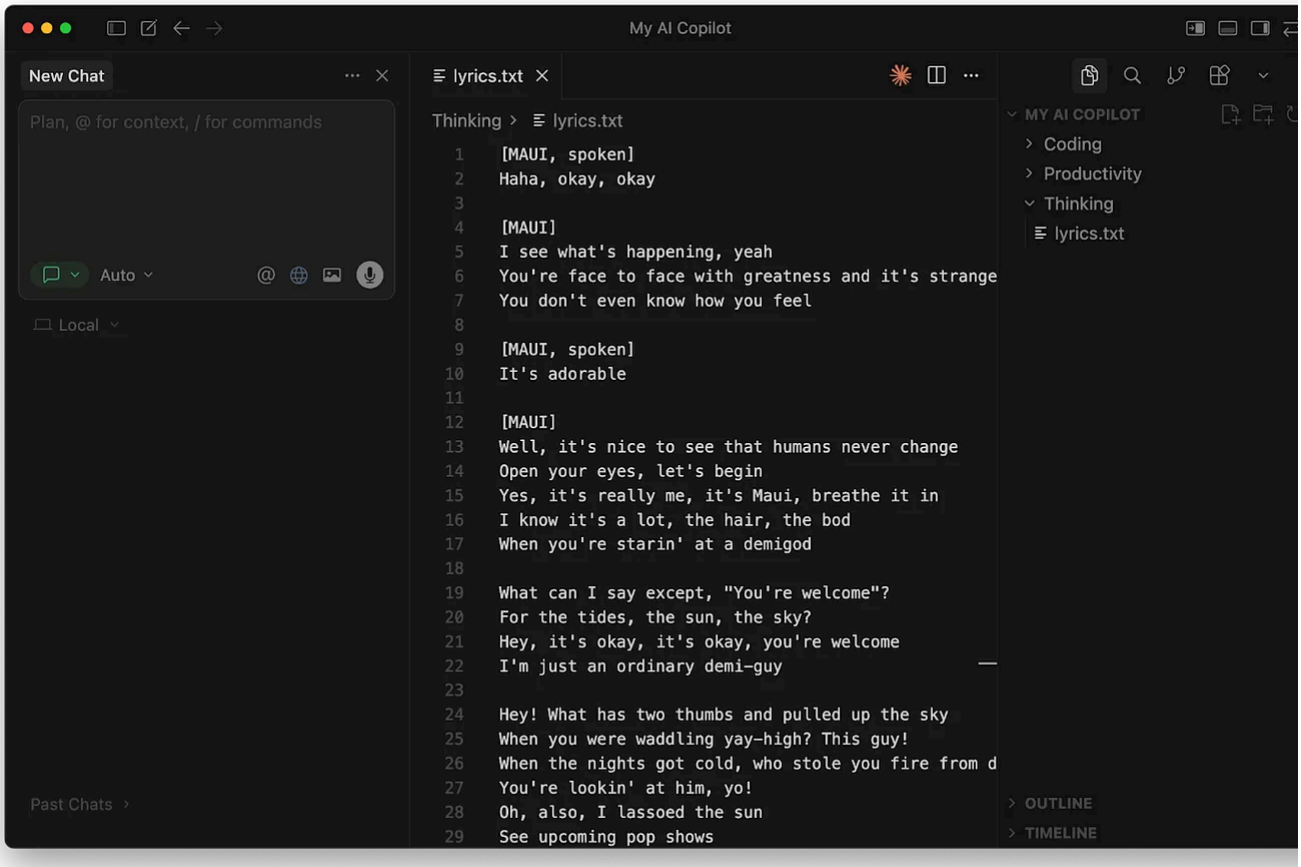
Name your file **lyrics.txt**:



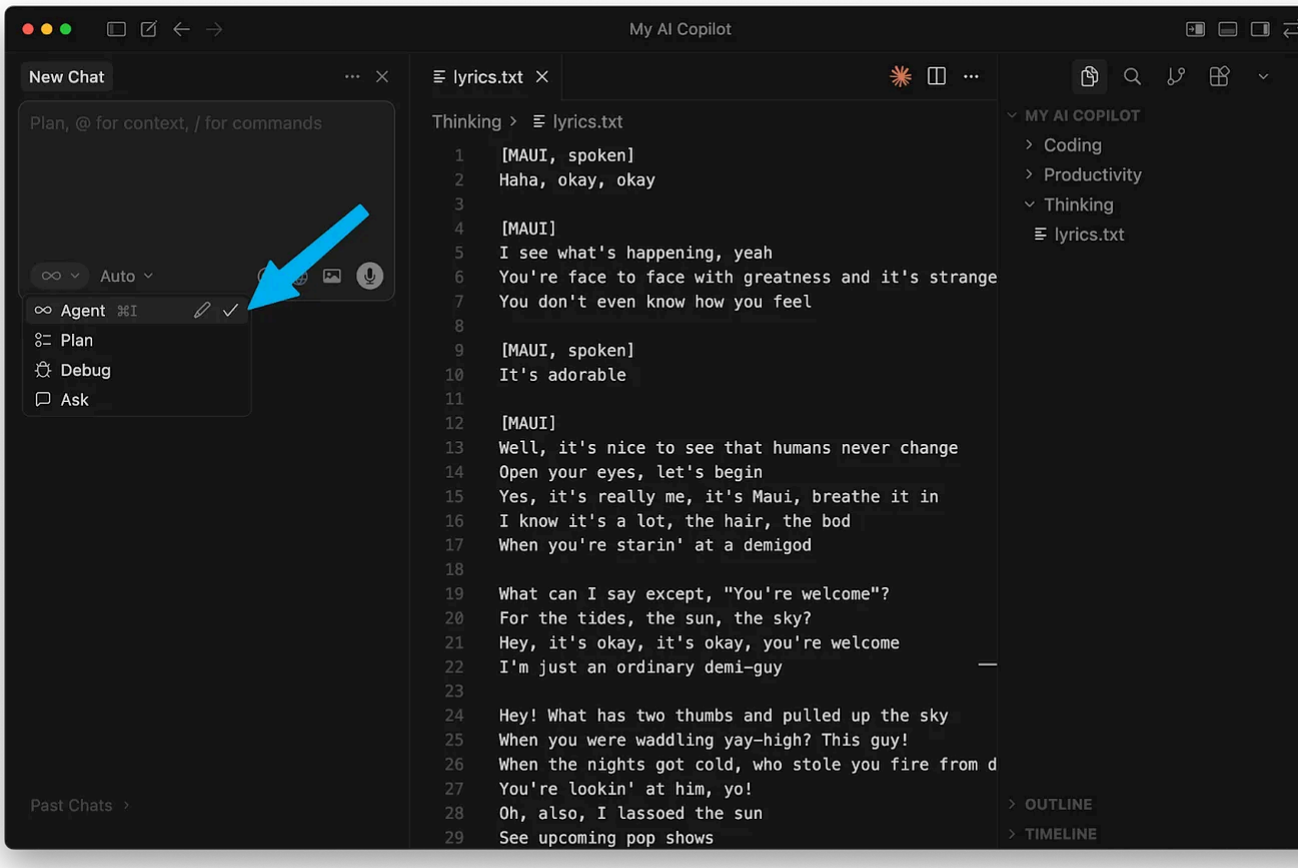
Next, search the web for your favorite Disney song (googling the title usually pri
lyrics). Copy the words to your clipboard:



Back in Cursor, paste them into the file you just created, and save the file:



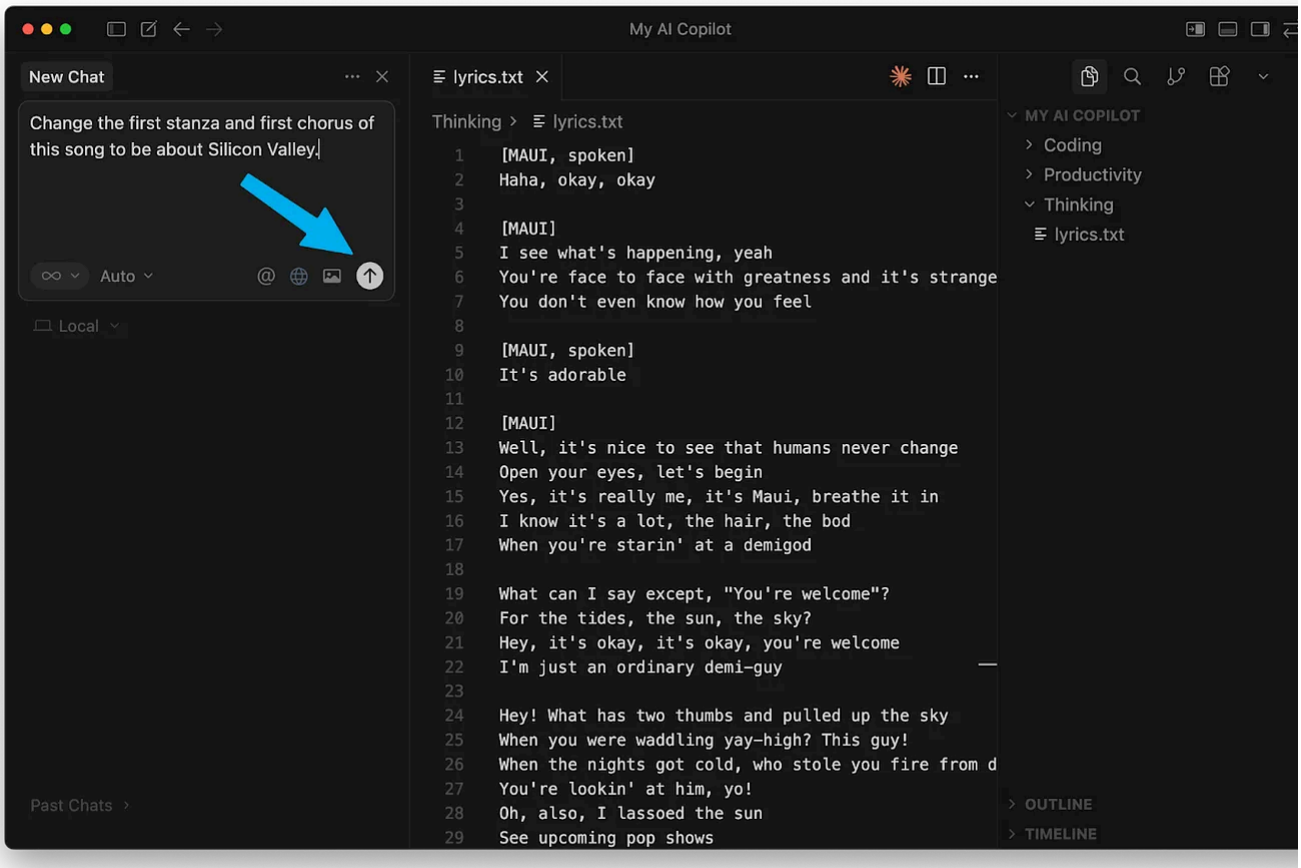
Next, switch to “Agent” mode in the chat box:



Finally, type in:

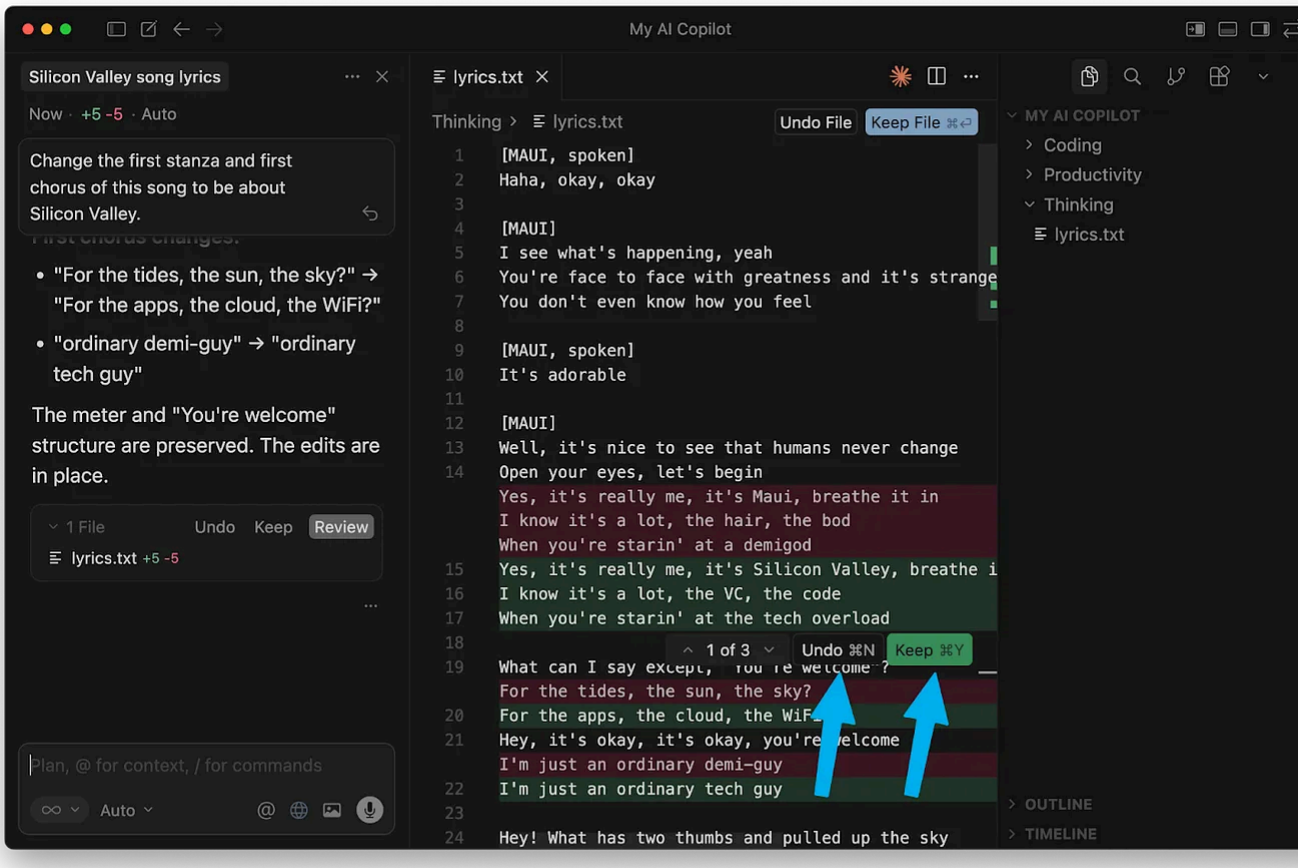
> `Change one line in the first stanza and one line in the chorus of lyrics.txt to be about Silicon Valley.`

and send it off using the “up arrow” button:



A lot just changed on our screen! You'll notice a lot of red and green in our lyrics file. Cursor modified our file. The red shows us each old line that it removed, and green shows us the new line that it added instead.

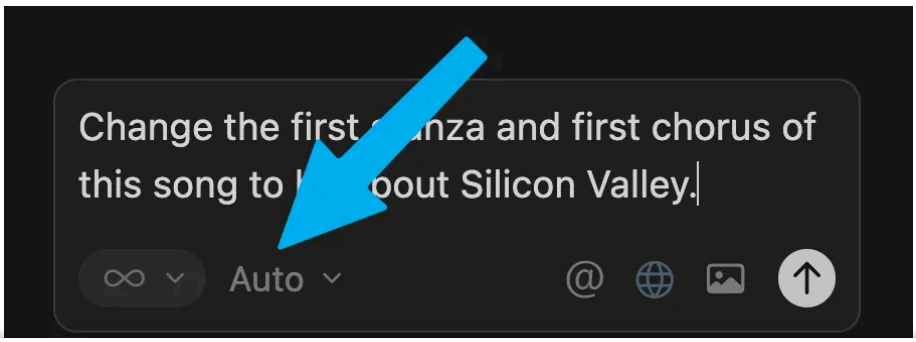
You can click "Undo" if you don't like the change, or "Keep" if you want it to ren



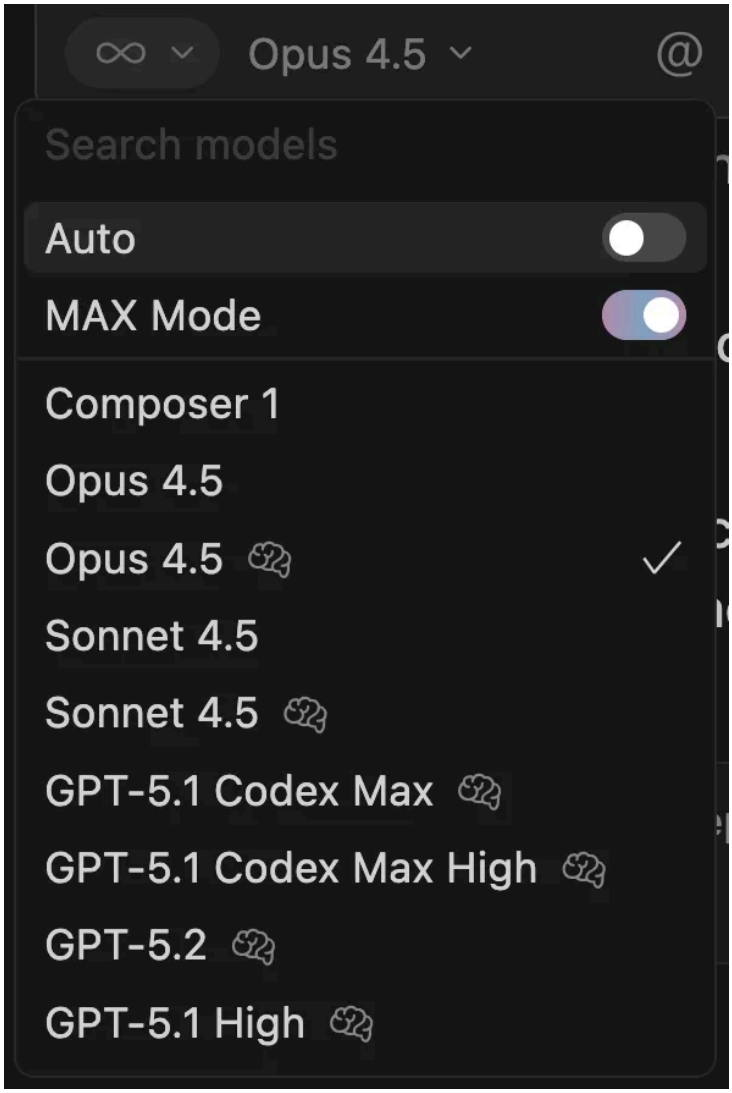
Step 5: See how different AI models behave

Now that you've made your first edit, let's explore a key product decision every A team faces: which model to use.

You'll notice there's another dropdown in our chat box:



Click into it and disable “Auto,” and take back the power to decide:



When we try the same query in multiple models, we build intuition for how each might tackle (or fumble) it differently. For example, Claude models are sensitive t copyright law and refuse to modify Disney songs (to get past this, change the enc your prompt to “...to make fun of the song itself,” which qualifies as “fair use”). W OpenAI’s models are less concerned with copyright, they stumbled when calling Cursor’s *apply_patch* tool, their preferred command for editing a text file (althoug is less common in the latest Codex models).

For writing, complex planning, and nuanced life advice (as of the time of this pos reach for Claude Opus. (We also found it's the best for experiencing this post fro inside Cursor.) Its cousin, Sonnet, is our workhorse for tasks involving lots of cor with a 1M token context window (and slightly faster responses).

Zooming out, there's a subtle lesson in Cursor's model dropdown: **There are only frontier LLMs, and they're available to all product teams.** Innovation is how we them.

Make a habit of switching models for tasks you care about. Over time, you'll deve genuine opinions about model tradeoffs—the kind of intuition that's hard to get benchmarks alone.

Step 6: Inspect your agent's tool calls

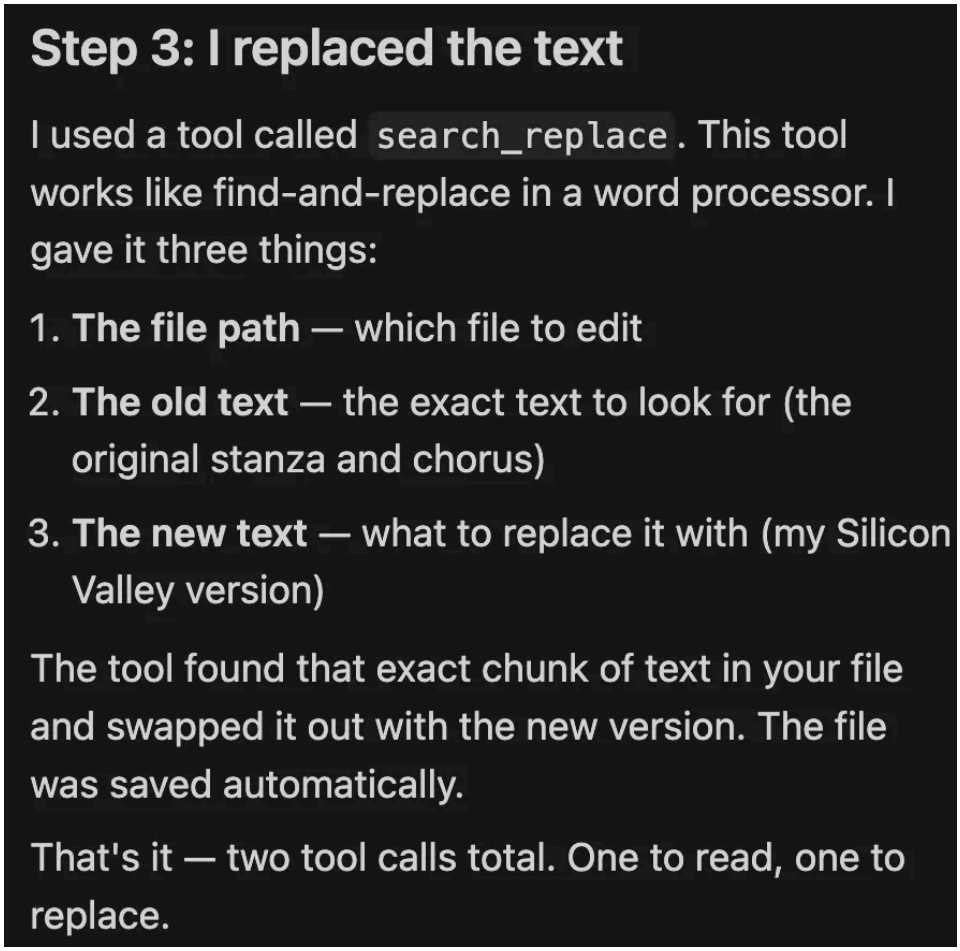
LLMs can only produce text, but when they take action (edit a file, fetch data, sea the web), they're calling tools. And *tool calling* is a distinct skill from everything e usually notice about LLMs.

Now ask your agent:

> `Can you walk me through each step (tool/thinking/reasoning/anything else) you used to accomplish this task?`

In our test, our LLM reported that:

1. It used a tool called *read_file* to find out what was inside the file
2. It thought about what to edit
3. It used a tool called *search_replace* to modify the text file

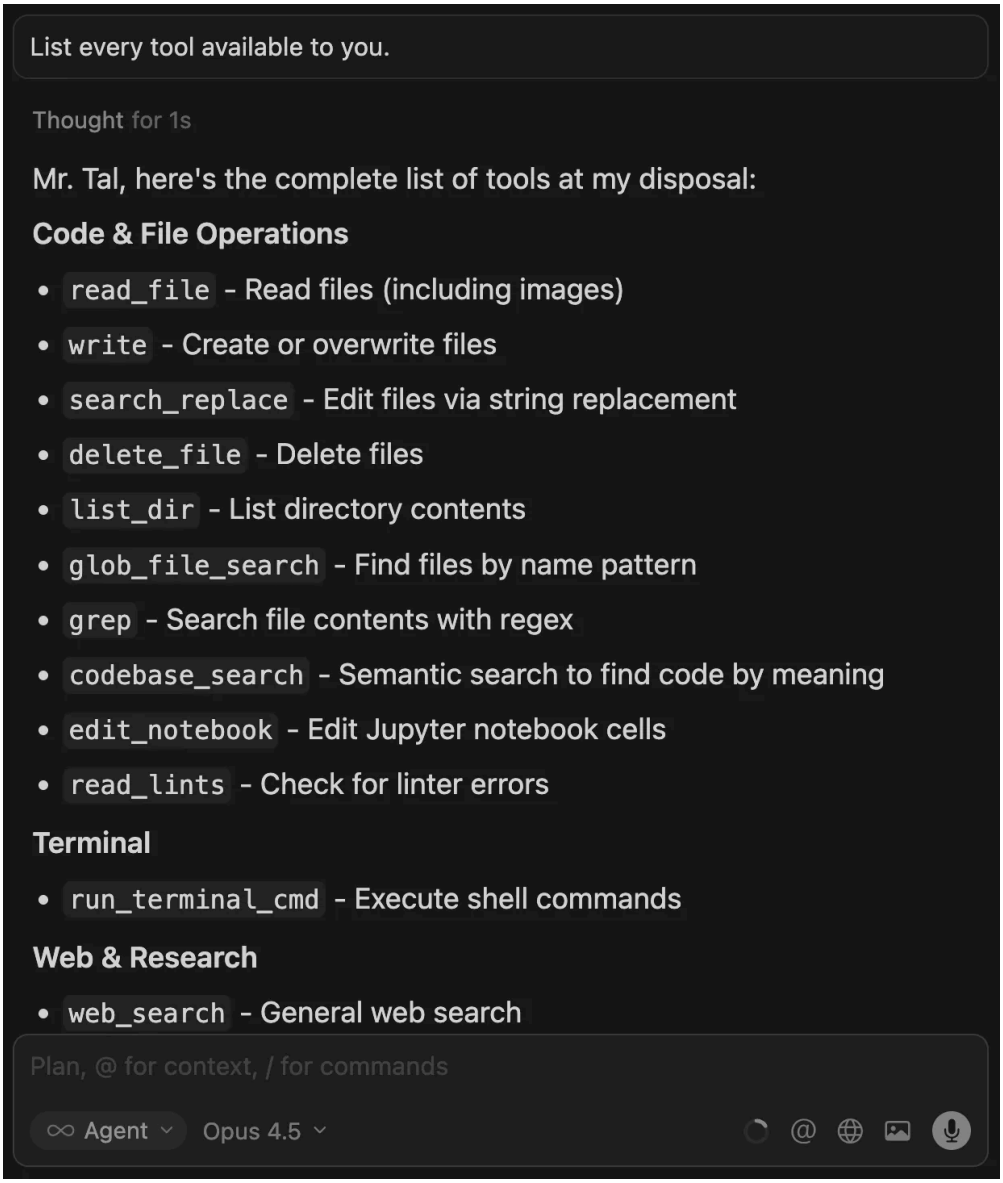


Don’t let this tool’s foreign name repel you. You’ve done “search and replace” ple times in Microsoft Word or Google Docs. And you’ve definitely “read a file” befo

(By the way, this is another place where models differ in approach! Gemini consistently accomplished this in three tool calls, while Opus used two. Try it ou see for yourself.)

Coding agents do most of their work with a small set of tools for file navigation a text editing. To see the full set, ask it:

> `List every tool available to you.`



Most of these tools have familiar names; you've definitely read the contents of a directory and deleted files before. Others, like *read_lints* and *run_terminal_cmd*, ar more common in software development (although coding agents can employ the non-technical requests too).

How does the LLM actually call the tool? An LLM can't run commands on your computer, so it relies on *Cursor* to do so. Think of it like hiring a handyman. The describes what it wants done, but it can't hold the hammer. Cursor is the handyn

LLM (i.e. a successful result or an error message) so the LLM can decide what to next. (If you've heard the terms "MCP client" or "agent harness," those both desc Cursor's role here.)

The way an LLM interacts with *tools* is eerily similar to how it interacts with *hun* If we view "classic ChatGPT" as a DM thread between an LLM and a human, the agents are a three-way group chat between an LLM, a human, and tools.

Now when someone asks, "Can our agent do X?" you'll instinctively think, "Wha would it need, and how good is our model at calling them?" This also connects b model selection: it's not just "smartest model wins." Tool calling is its own behav separate from reasoning or writing quality.

Wait, then what's the "MCP" I keep hearing about?

For most organizations, the most valuable data doesn't live in local text files but in external SaaS services. For an LLM to interact with Linear, Figma, Notion, Snowflake, BigQuery, Amplitude, or Mixpanel, those services need to provide th with *custom tools*.

Normally, each SaaS company would have to integrate a separate tool for each LI out there. To avoid this mess, the industry adopted a standard called Model Cont Protocol (MCP). That way, each SaaS company now only needs to build one conn that works everywhere.

If that sounds a lot like USB or Bluetooth, that's the right analogy. To continue tl comparison: most agent tools aren't MCP, just like most electrical wires aren't sh like USB plugs. **For simplicity, MCP is just another tool the agent can use, with standardized interface.**

Step 7: Put everything into practice by building your

We're going to build a very lightweight, minimal personal productivity system th organizes our contacts from various parts of our life, like notes, transcripts, and unstructured thoughts, as well as some tasks that we need to get done. (This lets temporarily ignore discovery, distribution, and pricing. We'll be free to focus on technically possible.)

By the end of this exercise, you'll be able to ask Cursor to create tasks from your backlog and get started on those tasks based on the context you provided in the knowledge and goals. In the process, we'll learn about RAG, memory, and contex engineering and build critical parts of product sense.

To get started, you can copy and paste the following prompt into Cursor (make s you're on "Agent" mode):

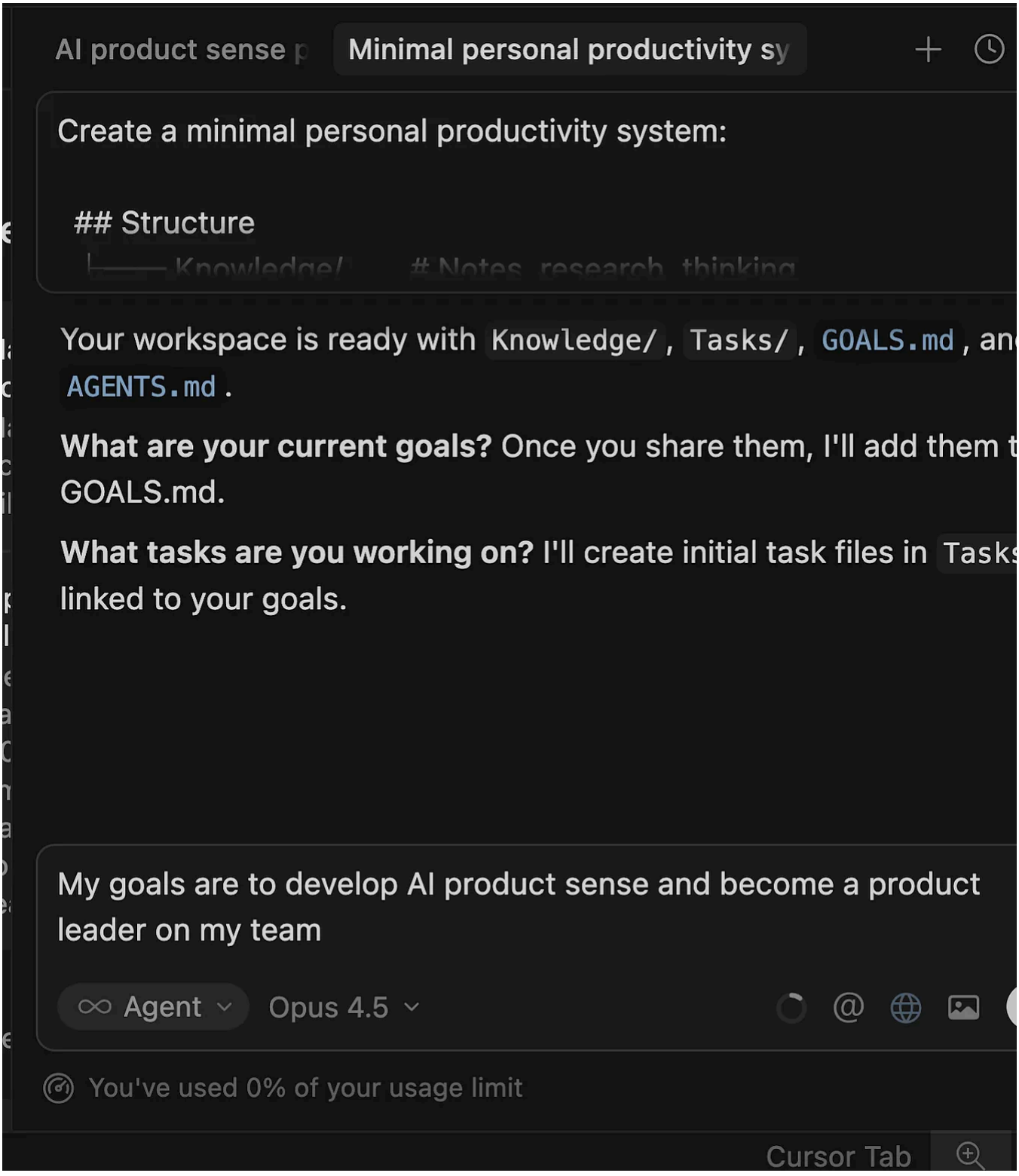
```
Create a minimal personal productivity system:

## Structure
├── Knowledge/ # Notes, research, thinking
├── Tasks/ # Action items as Markdown files
├── GOALS.md # Goals and priorities
└── AGENTS.md # AI assistant instructions

## AGENTS.md should instruct the AI to:
- Be a productivity assistant for goals and tasks
- Never write code — only Markdown
- Keep tasks tied to goals
- Suggest max. 3 daily priorities when asked
- Be direct and concise
```

```
2. Ask: “What are your current goals? Once you share them
I’ll add them to GOALS.md”
3. Ask: “What tasks are you working on? I’ll create initi
task files in Tasks/ linked to your goals”
4. Populate GOALS.md and Tasks/ from answers
```

(Optionally, you can clone the personal OS. *Hint*: you can copy and paste the link URL into your Cursor chat and ask it to clone the repo as well.)



Cursor should create two directories (folder): Tasks and Knowledge. It will also c a GOALS.md file, which should be your personal goals. At this point, Cursor wil

Congrats! You're now the PM of your own AI product. In the next few sections, use it to experience RAG, agent memory, and context engineering firsthand. Exc now you'll feel them instead of just reading about them.

Step 8: Kicking the tires of retrieval augmented generation (RAG) with your personal OS

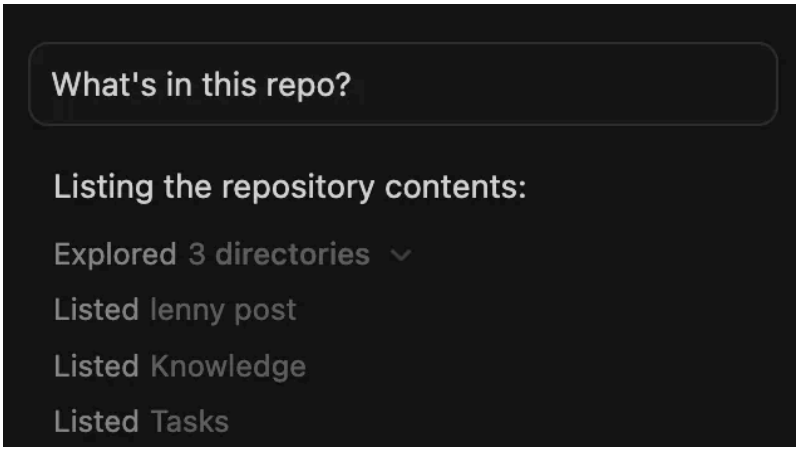
When an AI product gives wrong answers, the instinct is to blame the model. Be you reach for a larger model or expensive fine-tuning, ask: does the agent even h the right information?

Let's ask Cursor, "**What's in this repo? Explain it to me as a product manager:**" see what it responds with.

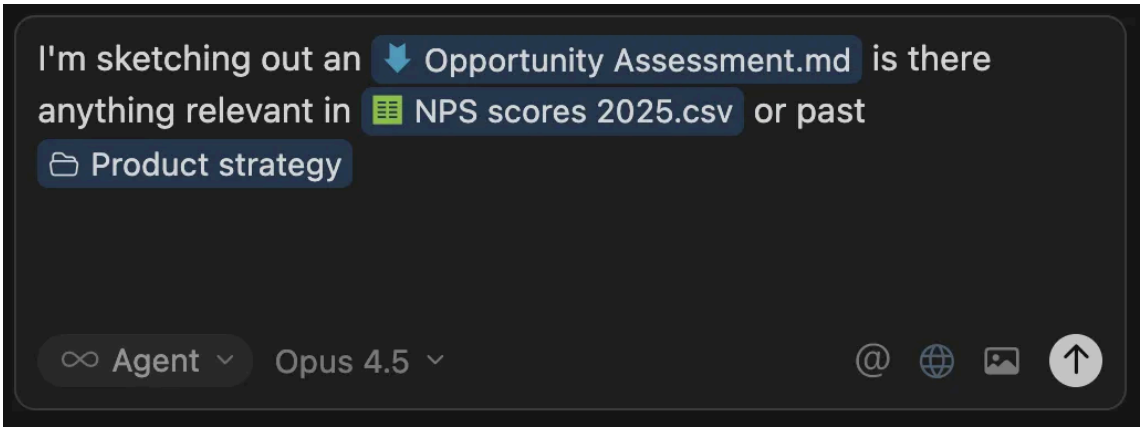
> 💡 This prompt is insanely powerful for just about any repo or folder you ope Cursor and Claude Code. We recommend starting here when opening any nev codebase.

RAG is a fancy term for "Before I start talking, I gotta go look everything up and it first." Despite the technical name, you've been doing it your whole life. Before answering a hard question, you look things up. Agents do the same.

In our case, our agent will search on top of your files and folders, to provide cont the LLM. There are a lot of different ways LLMs can search files within a codeba even find data from the internet to use in your response. At the end of the day, w it's Perplexity, Google AI Mode, Claude Code, Cursor, or anything out there you' seen with the term "RAG," it's some mechanism that pulls documents and starts chat with those documents.



You can also tag specific files within Cursor using the @ symbol, if you want the to reference specific pieces of context.



RAG fills the context window with what's relevant right now. But some context (
you are, how you like to work) should be there every time. That's where memory
in.

Step 9: Adding agent memory with AGENTS.md

LLMs are stateless, so we like to think of them as Mensa geniuses with the short
memory of a hamster. Every new chat thread is a blank slate, so if you want conti
you have to engineer it. As PMs, we have to ask: what should persist, and how?

All coding agents today follow a convention called AGENTS.md. This is a Markd file that, if found in any directory, is automatically appended at the top of every c thread.

(Quick disambiguation: AGENTS.md is not "subagents." Subagents are when one thread spawns another chat thread to handle a subtask. AGENTS.md is just instructions—it doesn't spawn anything. Think of AGENTS.md as a "sticky note every conversation" and subagents as "delegating to a colleague.")

Let that sink in: memory is just a text file prepended to every conversation. The no magic here. But that also means memory has a cost—whatever you put in AGENTS.md takes up context window space in every single chat. Too much mem and you have less room for everything else. This is why you want to be intentiona about what goes in memory (persistent, always relevant) vs. what you retrieve on demand with RAG (task-specific).

Examples of what you might find in a thinking partner's AGENTS.md:

- Who I am as a user and what this thinking partner is helping me with (e.g. "I product manager working at Acme Inc. on the platform team")
- How I want to work together (e.g. "Ask me questions to gain more context, f important missing information, and challenge my assumptions")
- Values to apply (e.g. "bias to action, make decisions with incomplete informa
- How to output (e.g. "write extremely succinctly, flat hierarchy, no bold or ita no em dashes")

By curating your own AGENTS.md, you start to feel what's actually useful vs. no And that's exactly the intuition you need when designing memory for your users.

Memory, RAG, tool definitions, conversation history—they all take up context. N

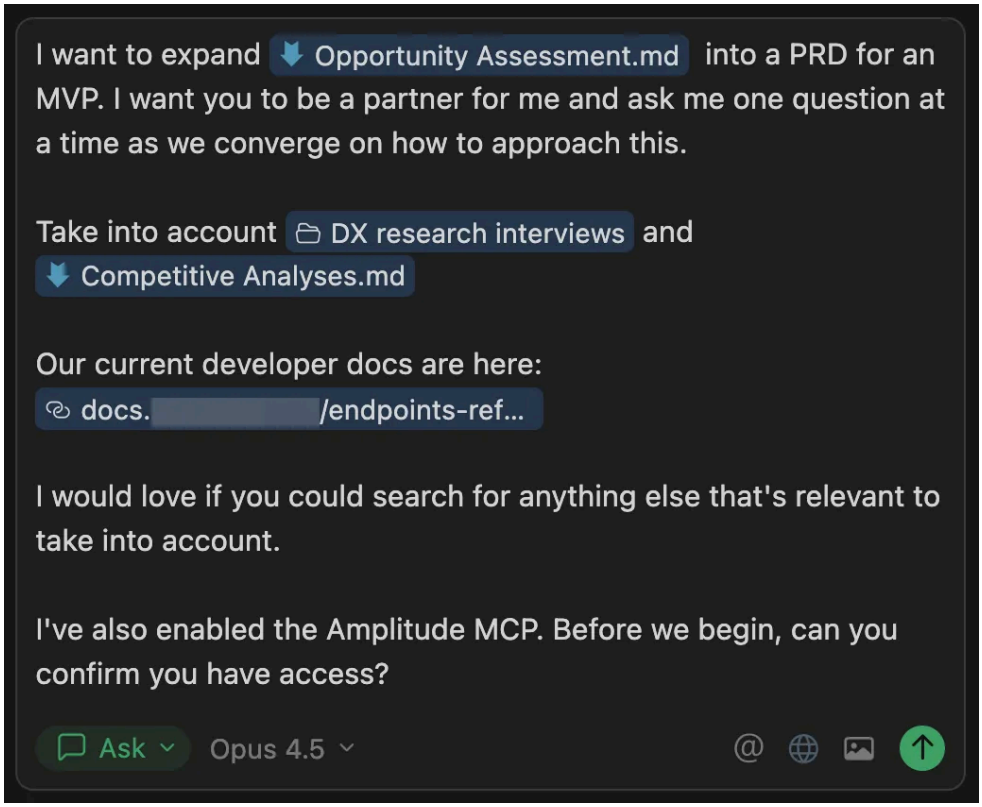
Anytime you've dragged a file into ChatGPT, said something that became part of memory, started a conversation with deep research, or created a project with spe instructions and knowledge, you were doing **context engineering**.

Even though you're using Cursor instead of ChatGPT, you'll still start each threa asking, "What context does this conversation need?" For example, writing a PRD might start by dragging and dropping any of the following into the Cursor windo

- A folder of user research transcripts
- An existing marketing landing page or sales deck
- An academic research paper or technical documentation
- Enabling an MCP tool connected to your analytics provider
- A saved set of instructions you want to apply
- A request to search for any other relevant information
- An area of your product's codebase

Try this out for yourself: Drag and drop a few PDFs, Markdown files of Google D or Word docs that you might be working on into this Knowledge folder in Curso ask Cursor to expand on one of the topics or ask you questions about it before ge to work.

Putting it all together, you might start a Cursor chat thread this way:

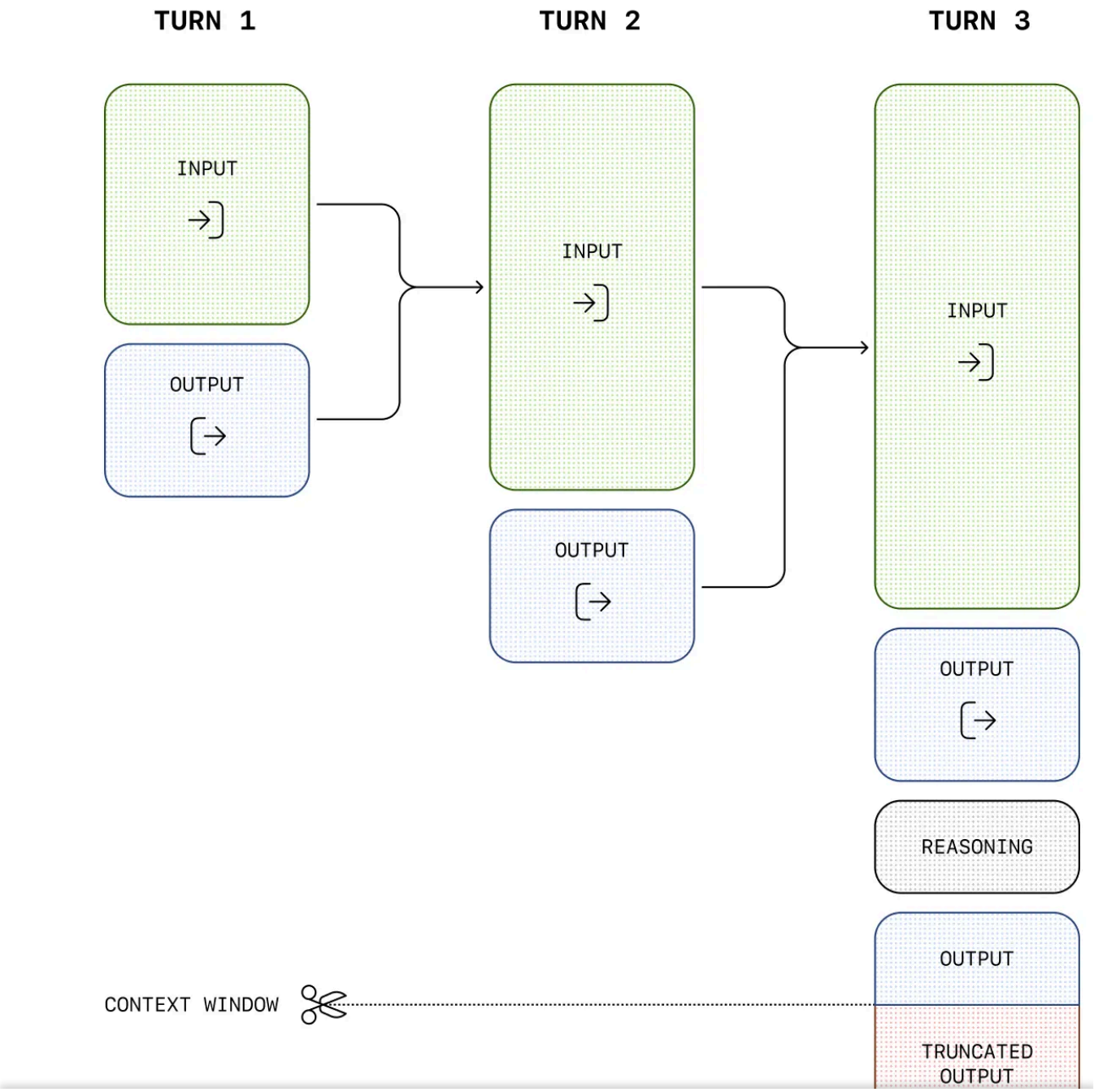


Prompt for Cursor:

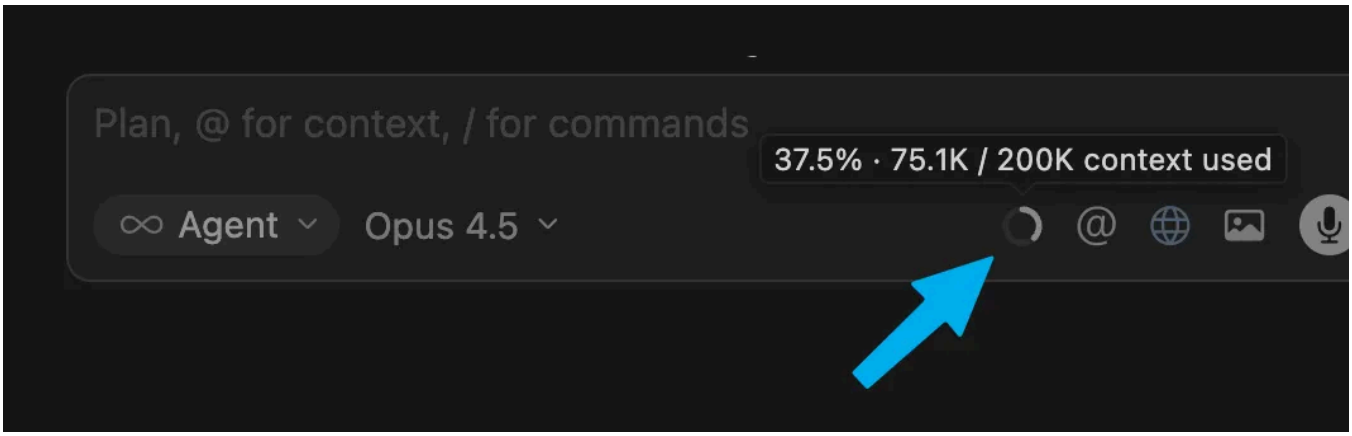
> I want you to expand on this @file (replace with your act file, like this post). I want you to be a partner for me ask me one question at a time as we are converging on how approach this. I would love it if you could search for anything else that's relevant to this document. You can a create some follow-up ideas for me to explore and turn th into tasks.md files in the Tasks folder, based on our discussion.

Context engineering happens in two ways in Cursor: (1) what you choose to inclu each time you start a chat (dragging files, enabling tools) and (2) what gets automatically included every time (like AGENTS.md, MCPs, or tools). That's a lo

In a conversational LLM chatbot (like ChatGPT or Claude), context increases wi
each turn of the conversation as the LLM's last response becomes part of the nex
turn's input. If you've ever gotten a notification that you hit the limit of your cha
you've probably hit a context window:

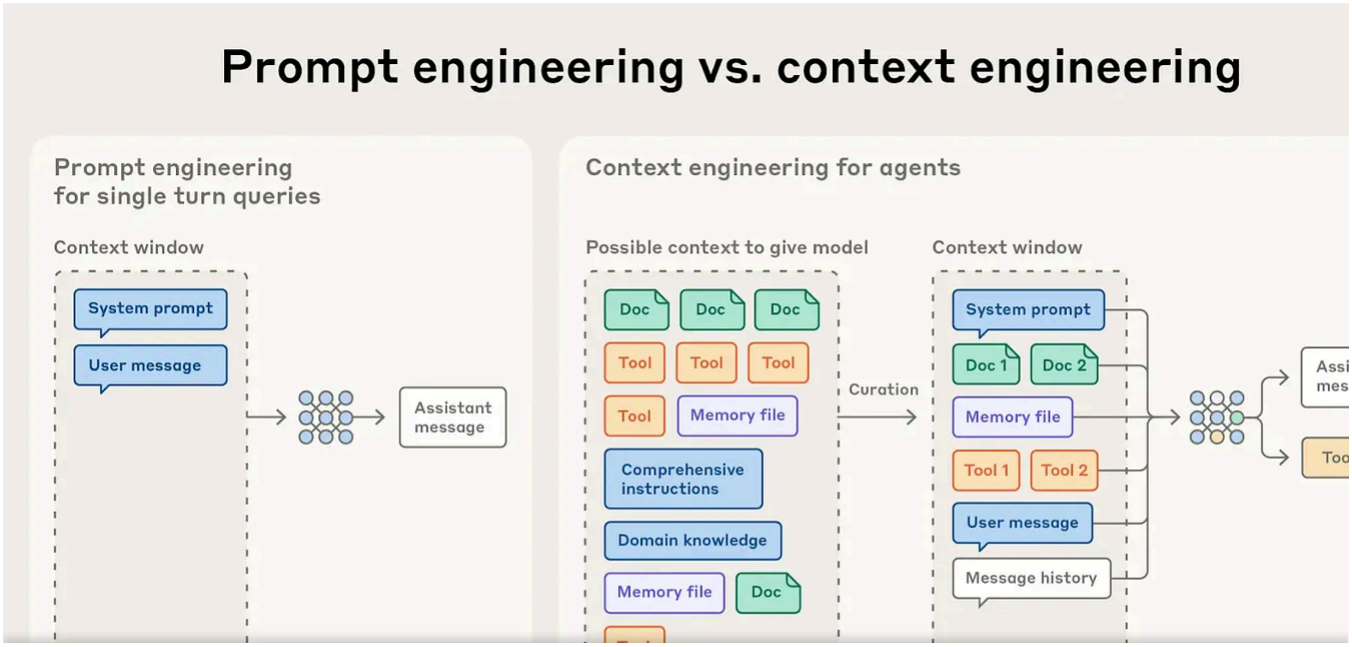


Coding agents are delightfully transparent about how much of your context winc you've used. In Cursor, hover your mouse over the little pie chart just beneath th box (you might have to increase the width of the chat pane to see it).



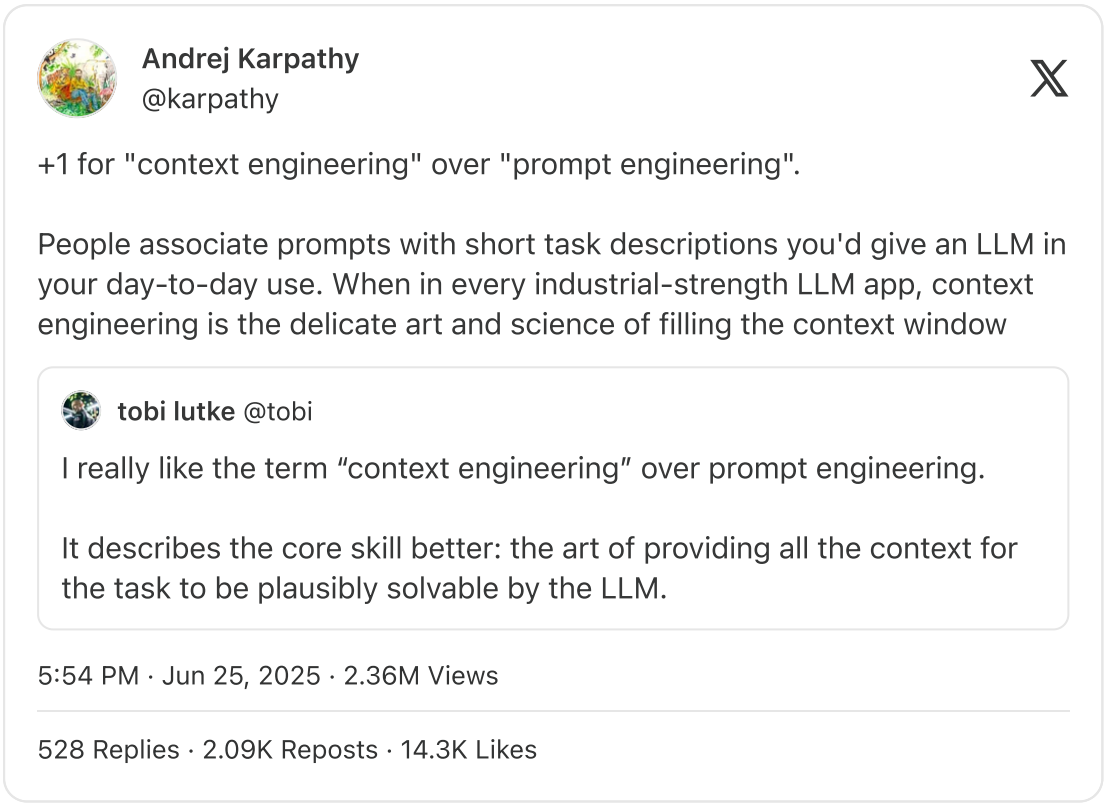
We watch it like a car's gas tank. This helps us intuitively understand what a 200 token or 1M token context window actually feels like, and how quickly it actually out. The best way to understand a specific car's gas mileage is to drive that car.

In an agentic LLM application, the context window might contain a whole lot m



Does Anthropic's diagram now feel familiar? Even though it's about AI applicati
it's remarkably similar to our earlier example of writing a PRD. How cool is that

When you set up AI to do its best work within its limited context window, you're
"context engineering."



Tool definitions, RAG, memory, goals: they all compete for the same limited con
window. How you fill that window determines how well your agent performs. Th
the same tradeoff every AI engineering team faces when building production ap
except now you're feeling this problem firsthand. There's no perfect formula for
What goes in depends on the task, the model, and what you're trying to achieve.
develop intuition through trial and error (and regularly using your personal OS).

Watch out for "context rot"

As you use more and more context, you'll notice that your threads may get dumb

Context rot is a fuzzy phenomenon. How badly you feel it will depend on the tas hand, as well as the particular model's training. You'll feel context rot (and degra performance) less in use cases like creative brainstorming and more in precision financial, coding, or data analysis.

The best way to get a feel for context rot is by using an LLM for tasks you genuir care about. Try to keep an eye on Cursor's token counter and, as it gets fuller, no the quality of the outputs. With more context in a thread, is the agent getting be worse? What if you try a new thread? We found this to be a fun, addictive game, of like driving a hybrid car and noticing when it's switching from battery to gaso Along the way, you build an intuition that goes further than any academic graph

Conclusion

For us, the shift to really, truly understanding AI products happened when we w agents work day after day. Cursor allowed us to observe an AI model meander th a task: read our context, run RAG, try a tool call, hit an error, reason, try another .

That's when it clicked: Cursor is just an AI product like any other, composed of t tools, and results flowing back into more text—except Cursor runs locally on our computer, so we can watch it work and learn. Once we were able to break down a product into these same building blocks, our AI product sense came naturally. N that you've read this post, you can too.

When something impressive drops, you can now take a breath and decompose it. agent that plays Settlers of Catan for 75 minutes. Granola's eerily personalized ye review. A lobster-themed Claude Code with full control of your computer that ar can set up. You can simply ask yourself: How would I reproduce that inside Curs I had to "Wizard of Oz" it on my computer, what familiar parts would I assemble

what just became feasible. In Paul Buchheit's words, when you truly understand products, you'll "live in the future."

Thanks, Tal and Aman! For more, check out their in-depth workshop Build AI Product S (starting next week, get 15% off using this link) and their upcoming free Lightning Lesso to Know What AI Products to Build, in partnership with my one of my other favorite collaborators, Hilary Gridley. You can also book Tal and Aman for a build sprint with y team.

Have a fulfilling and productive week 🙏

If you're finding this newsletter valuable, share it with a friend, and consider subscribing if you haven't already. There are group discounts, gift options, and referral bonuses available.

Sincerely,

Lenny 👋

---

1 With an eye for building AI products at scale, we like to keep a pulse on what smaller, cheaper, faster models can do. Once in a while, we'll re-run a task in Claude Haiku, G Flash, or GPT mini to feel their speed, tool-calling skills, judgment, and personality. Switching models for tasks we truly care about helps us catch important nuances and teaches us more than any benchmarks or buzz.

2 To avoid packing everything into the context window up front, AI engineers constant devise all sorts of creative solutions. Here are just a few:

- "Progressive disclosure" is the art of pulling in frameworks, tools, or instructions when they're needed instead of up front. This can happen manually (like when w

context window stays a bit cleaner. (This is similar to a tool call, with the job bei done by an LLM.)

- Condensing MCP tools into a single tool, or giving the agent freedom to select (a combine) tools on the fly.
- Pruning or summarizing the context window as you near its limit. If you've ever of room in a chat thread and asked the LLM to summarize so you can start a fres thread—you've already done this yourself.

These strategies allow AI applications to break out of the zero-sum game of context li to keep agents running autonomously on complex tasks for as long as possible. This is where AI engineering gets fun and creative. The best part is that no one has fully figu this out, and ideas are constantly evolving.

568 Likes · 48 Restacks

← Previous Nex

A guest post by
Tal Raviv
Early @ Patreon, Riverside, Wix, AppsFlyer, DuckDuckGo

Subscribe

A guest post by
Aman Khan
AI Product Guy

Subscribe to A

Discussion about this post

Comments Restacks

Write a comment...

Jack Cohen Flow State 3 Feb

Liked by Lenny Rachitsky, Tal Raviv, Aman Khan

This "article" creates a whole new paradigm for learning: interactive, memorable, persona

Actually going through the steps in Cursor left me feeling like Neo in The Matrix, "Now I kr fu [AI]."

(Except now I'm learning to create agents instead of kill them...)

LIKE (19) REPLY

1 reply

Hamel Husain Hamel's Substack 3 Feb

Liked by Lenny Rachitsky, Tal Raviv, Aman Khan

Excellent work Tal and Aman 🙌

LIKE (9) REPLY

2 replies

37 more comments...

Cookie Policy

We use cookies to improve your experience, for analytics, and for marketing. You can accept, reject, or manage your preferences. See our privacy policy.